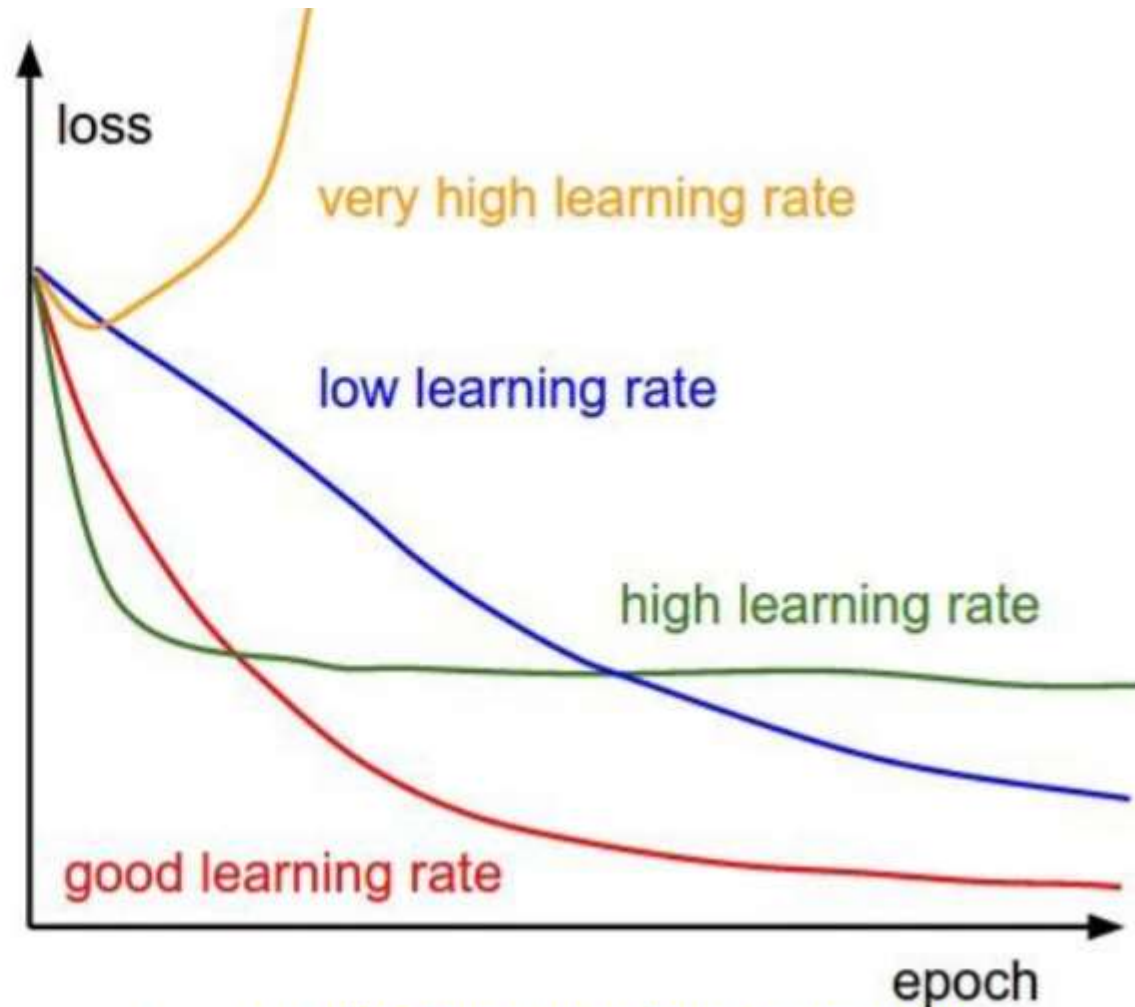


Training Neural Networks

Learning Rate: Implications

- What if too high?
- What if too low?

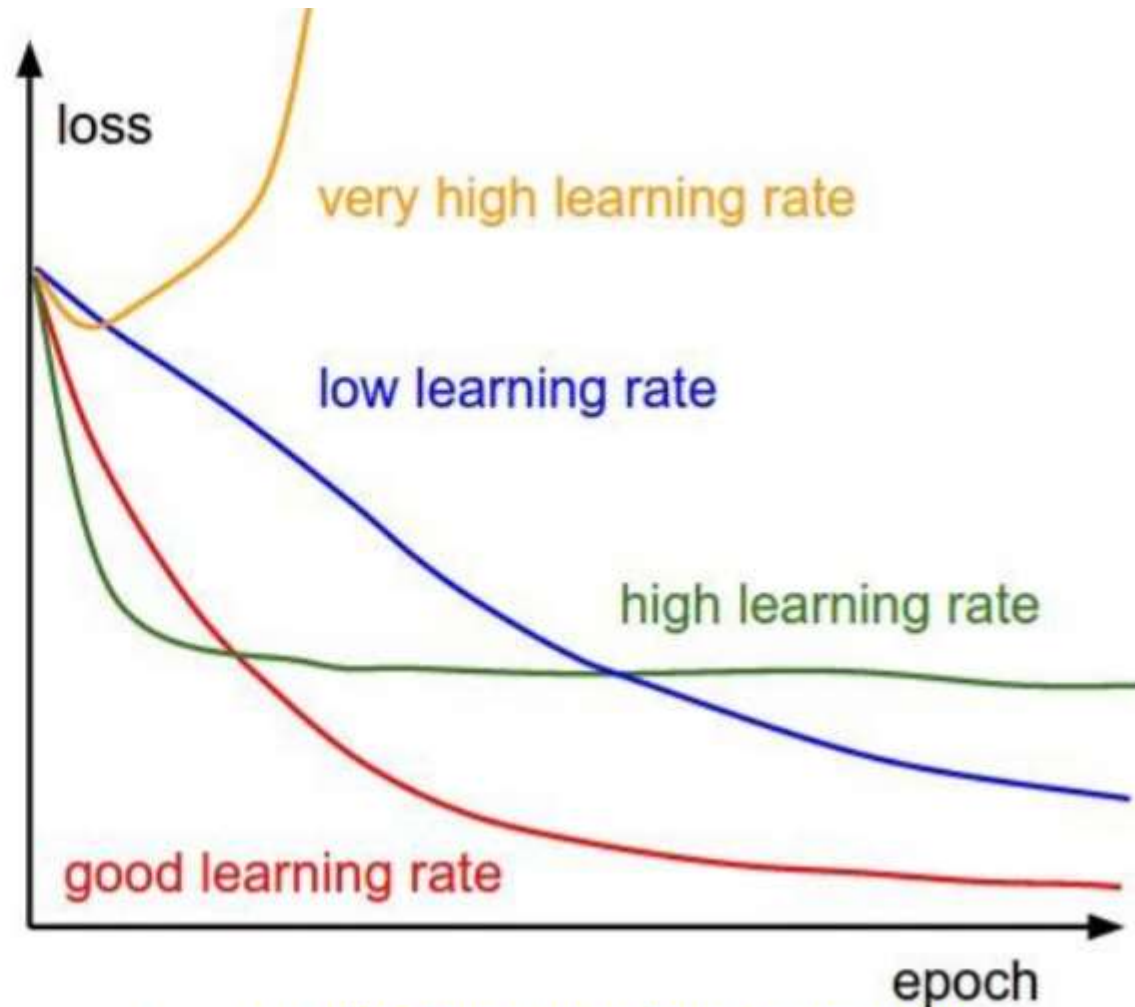


Source: <http://cs231n.github.io/neural-networks-3/>

Learning Rate: Implications

- What if too high?
- What if too low?

Q: Which one of these learning rates is best to use?



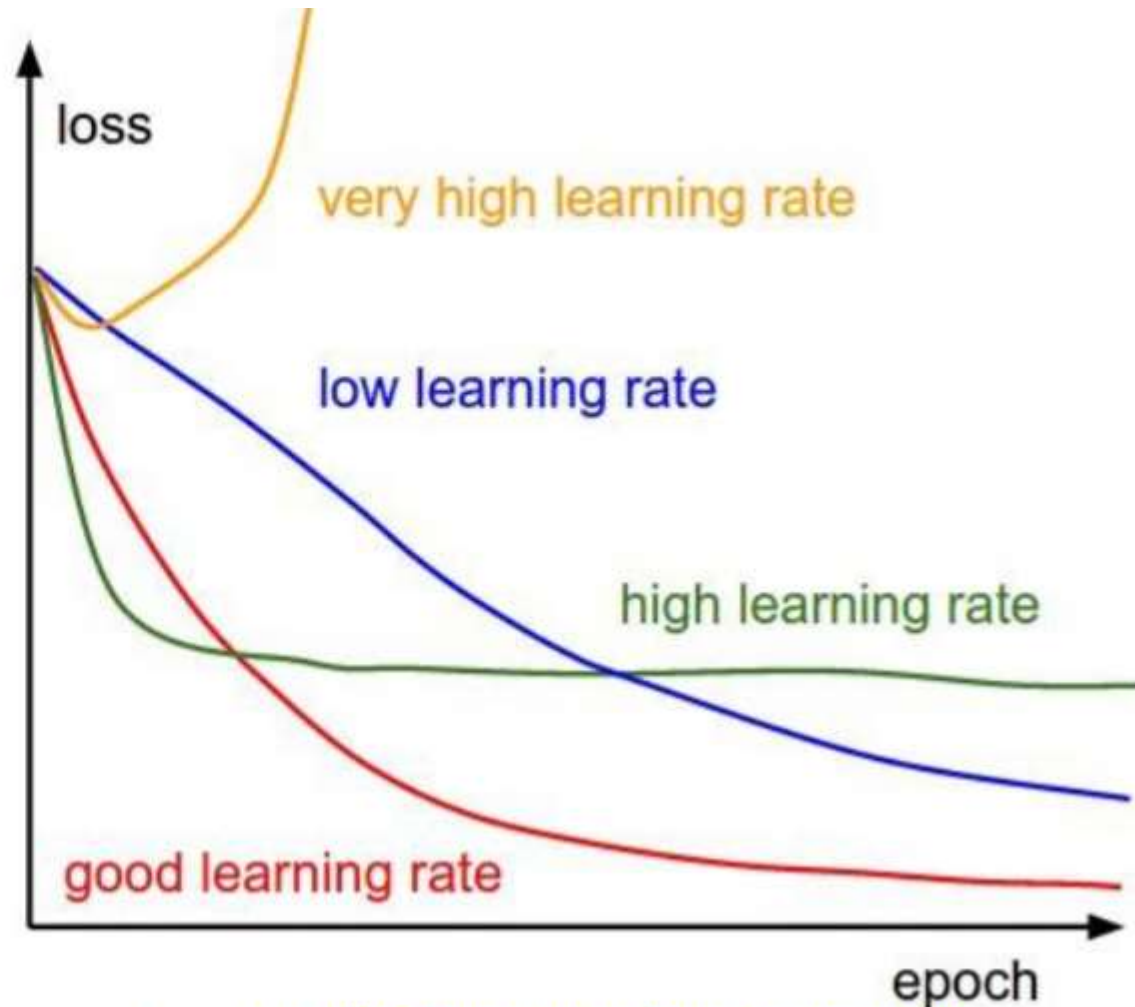
Source: <http://cs231n.github.io/neural-networks-3/>

Learning Rate: Implications

- What if too high?
- What if too low?

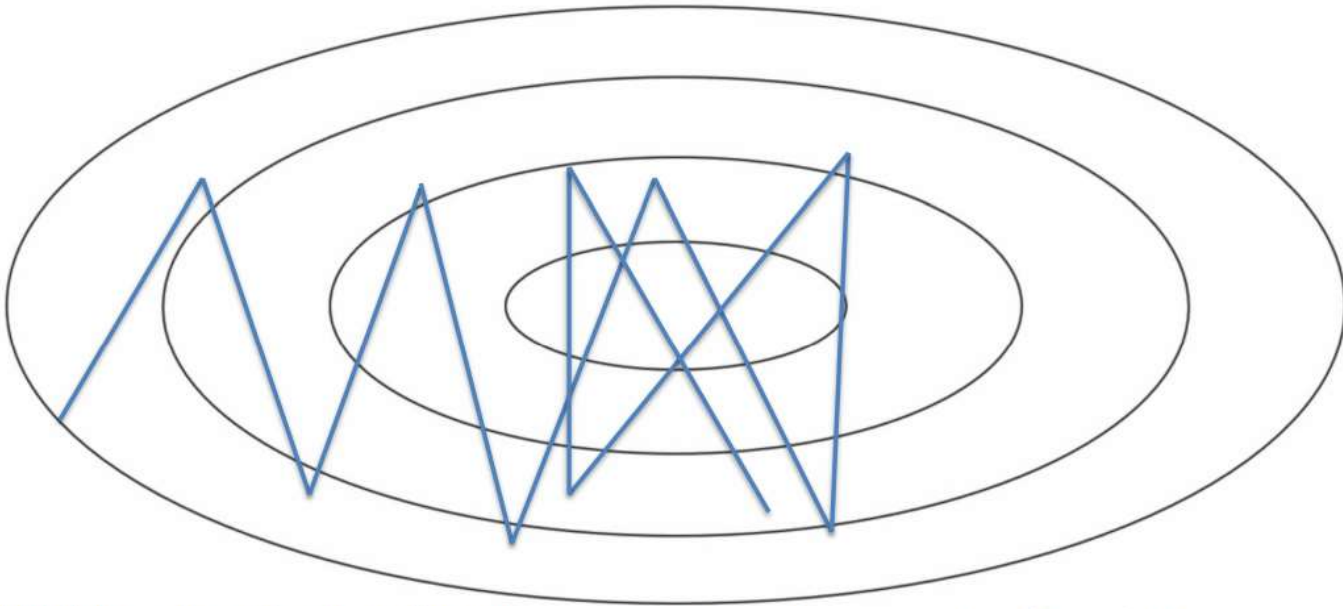
Q: Which one of these learning rates is best to use?

A: All of them! Start with large learning rate and decay over time



Source: <http://cs231n.github.io/neural-networks-3/>

Learning Rate



Need high learning rate when far away

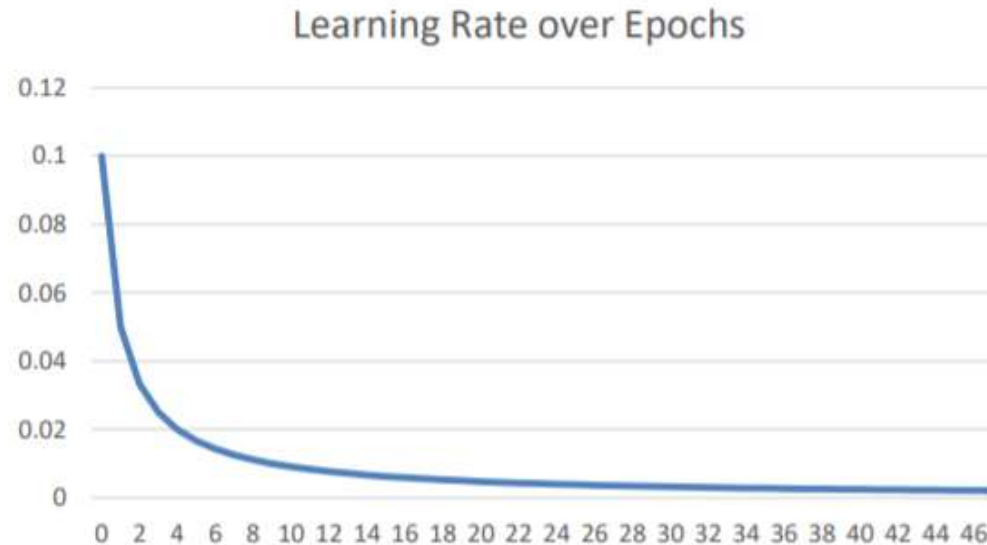
Need low learning rate when close

Learning Rate Decay

$$\alpha = \frac{1}{1 + \text{decay_rate} * \text{epoch}} \cdot \alpha_0$$

– E.g., $\alpha_0 = 0.1$, $\text{decay_rate} = 1.0$

→ Epoch 0: **0.1**
→ Epoch 1: **0.05**
→ Epoch 2: **0.033**
→ Epoch 3: **0.025**
...



Learning Rate Decay

Many options:

- Step decay $\alpha = \alpha - t \cdot \alpha$ (only every n steps)
 - T is decay rate (often 0.5)
- Exponential decay $\alpha = t^{epoch} \cdot \alpha_0$
 - t is decay rate ($t < 1.0$)
- $\alpha = \frac{t}{\sqrt{epoch}} \cdot \alpha_0$
 - t is decay rate
- Etc.

Training Schedule

Manually specify learning rate for entire training process

- Manually set learning rate every n-epochs
- How?
 - Trial and error (the hard way)
 - Some experience (only generalizes to some degree)
- Consider: #epochs, training set size, network size, etc.

Basic Recipe for Training

- Given ground dataset with ground labels
 - $\{x_i, y_i\}$
 - x_i is the i^{th} training image, with label y_i
 - Often $\text{dim}(x) \gg \text{dim}(y)$ (e.g., for classification)
 - i is often in the 100-thousands or millions
 - Take network f and its parameters w, b
 - Use SGD (or variation) to find optimal parameters w, b
 - Gradients from backpropagation

Learning

- Training set (*'train'*):
 - Use for training your neural network
- Validation set (*'val'*):
 - Hyperparameter optimization
 - Check generalization progress
- Test set (*'test'*):
 - Only for the very end
 - NEVER TOUCH DURING DEVELOPMENT OR TRAINING

Learning

- Typical splits
 - Train (60%), Val (20%), Test (20%)
 - Train (80%), Val (10%), Test (10%)
- During training:
 - Train error comes from average minibatch error
 - Typically take subset of validation every n iterations

Basic Recipe for Machine Learning



Example scenario

Ground truth error 1%

Training set error 5%

Val/test set error 8%

Basic Recipe for Machine Learning



Example scenario

Ground truth error 1%

Training set error 5%

Val/test set error 8%



Bias
(underfitting)

Basic Recipe for Machine Learning



Example scenario

Ground truth error 1%

Training set error 5%

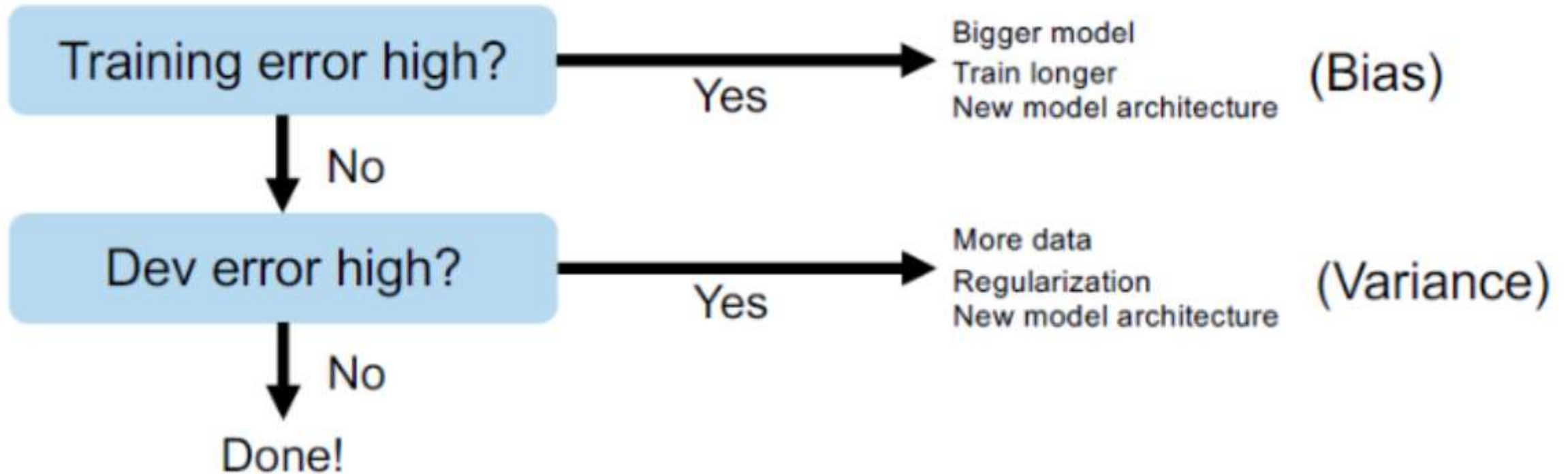
Val/test set error 8%



Bias
(underfitting)

Variance
(overfitting)

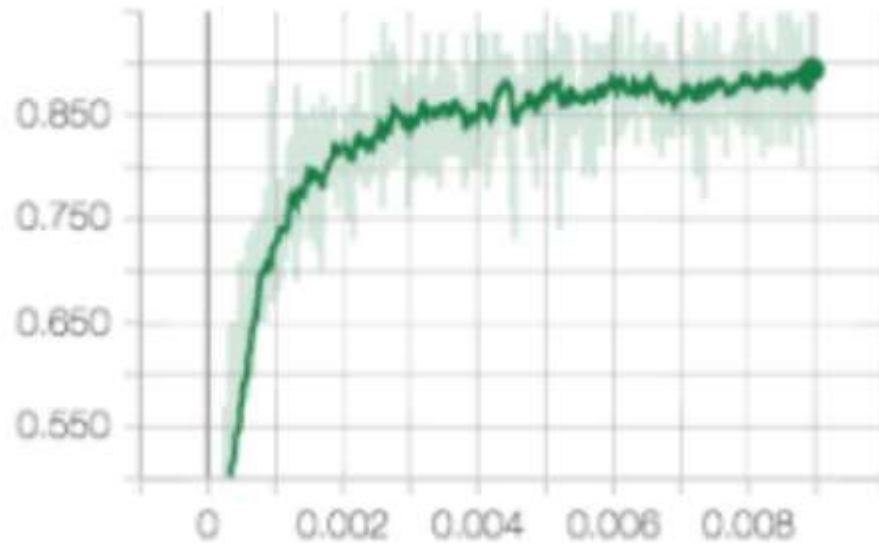
Basic Recipe for Machine Learning



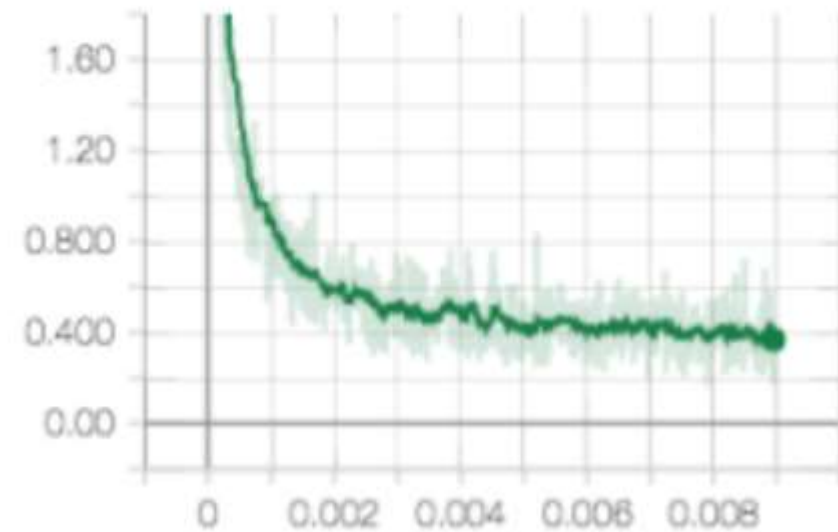
Credits: A. Ng

Learning Curves

- Training graphs
 - Accuracy

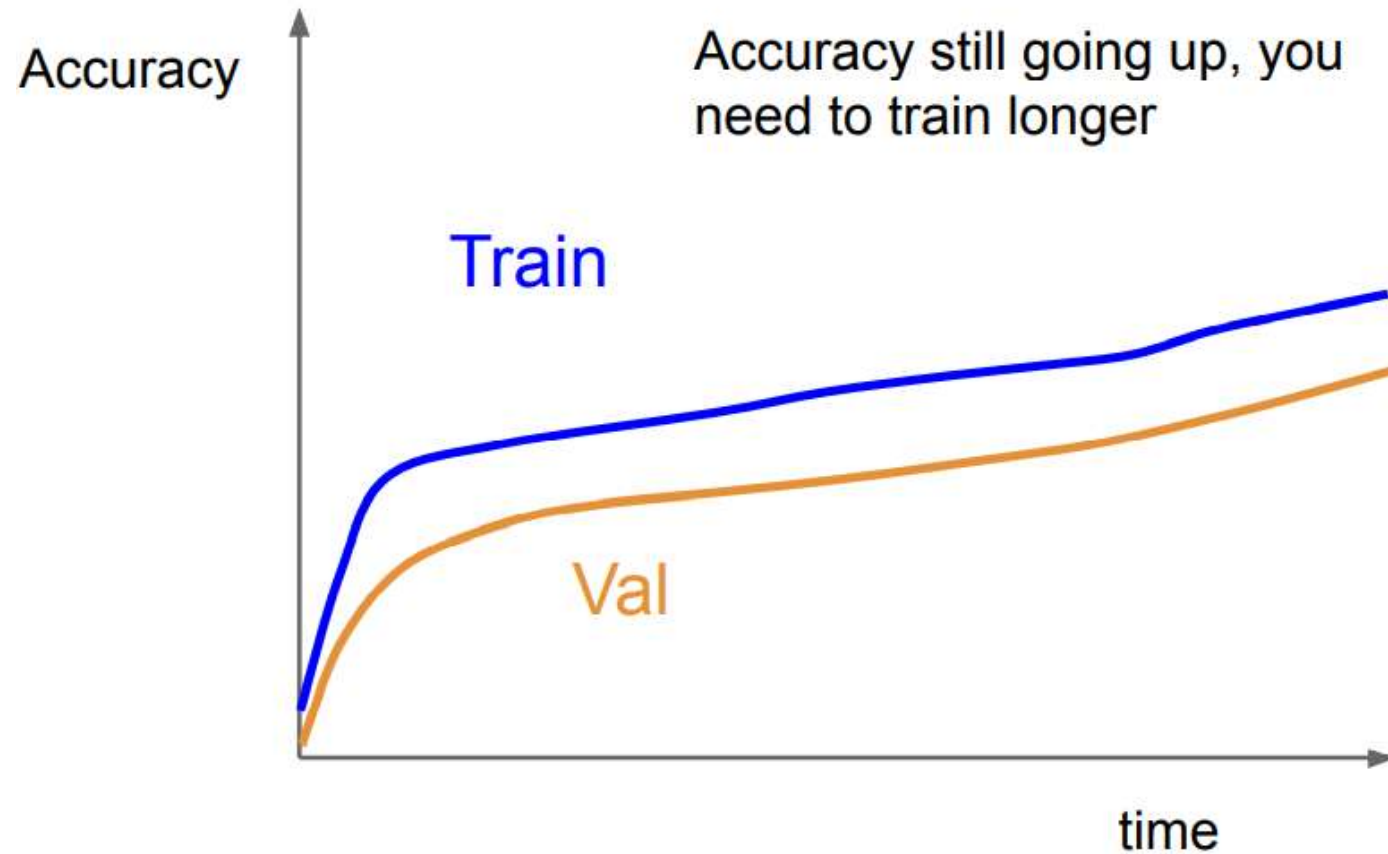


- Loss

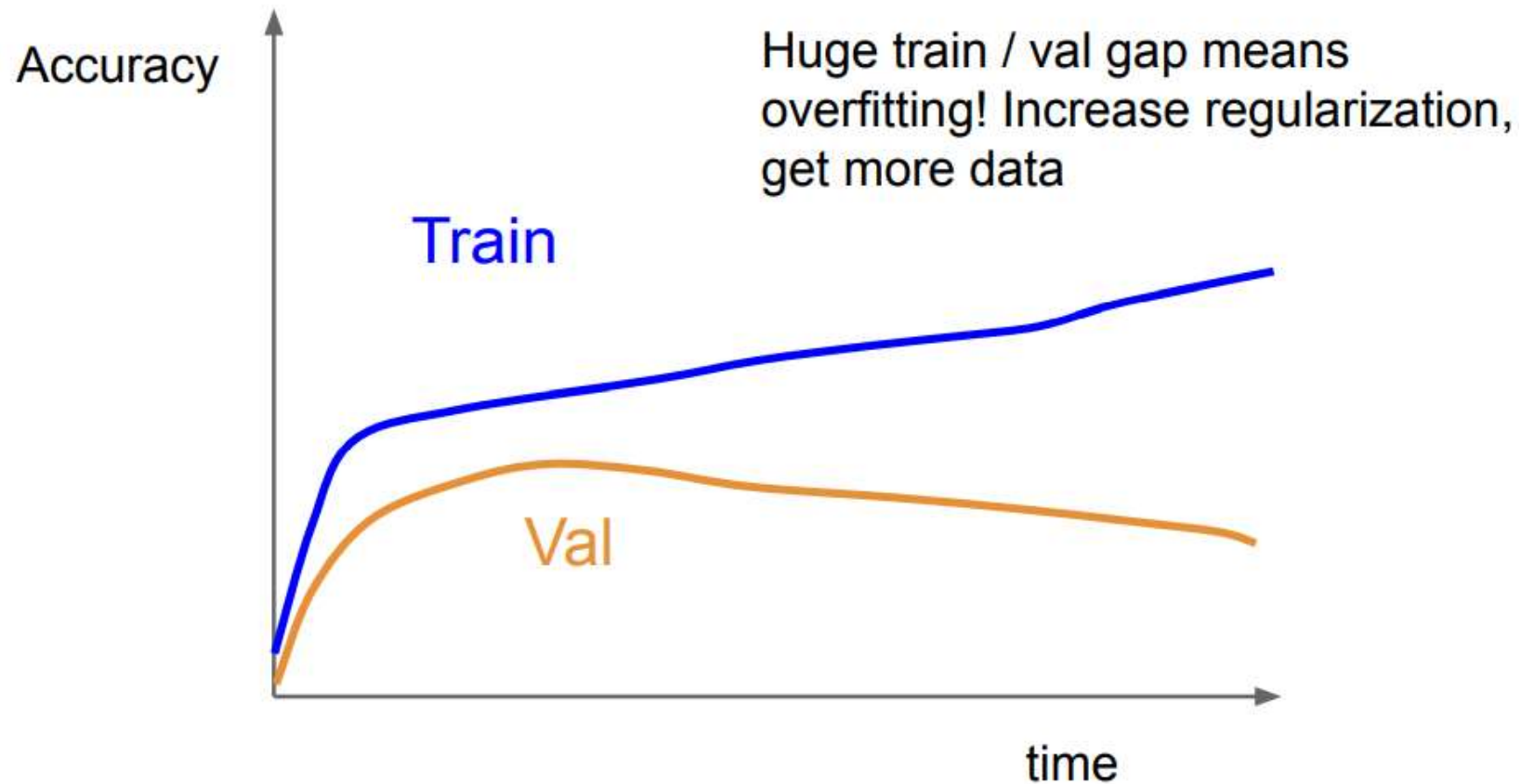


(EMA smoothing)

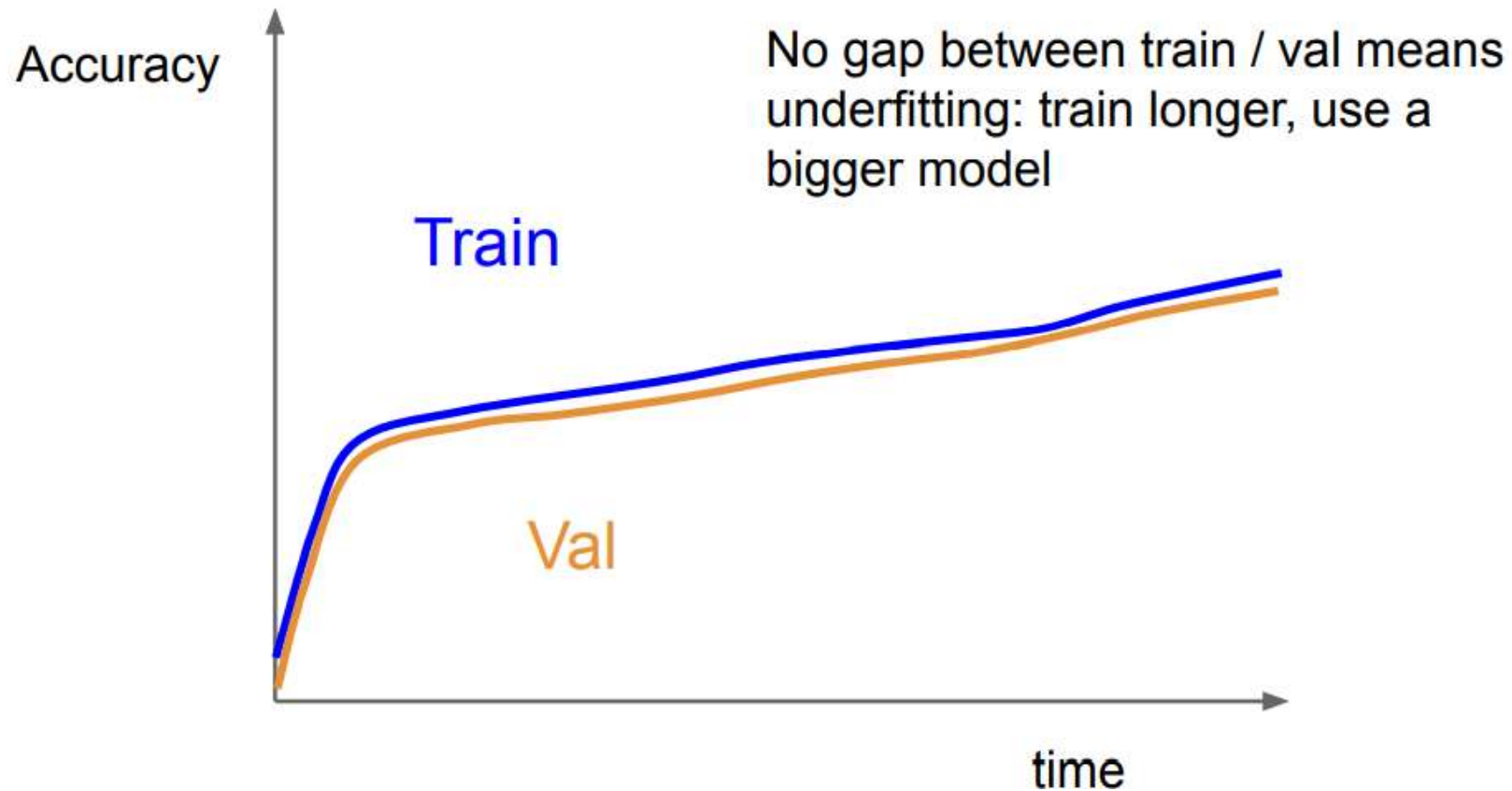
Learning Curves



Learning Curves



Learning Curves

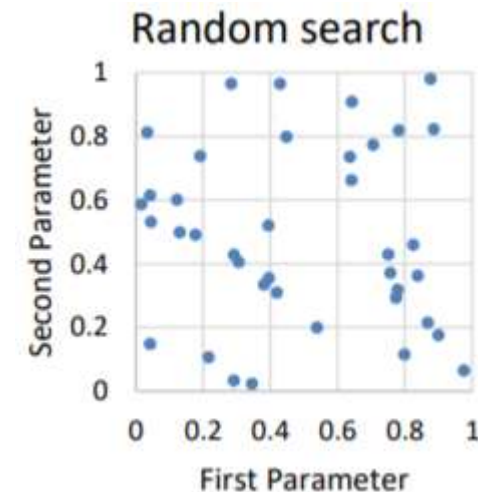
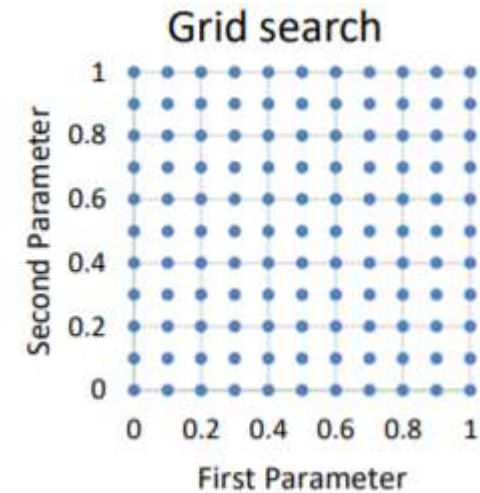


Hyperparameters

- Network architecture (e.g., num layers, #weights)
- Number of iterations
- Learning rate(s)
- Regularization
- Batch size
- ...

Hyperparameter Tuning

- Manual search
- Grid search
 - Define ranges for all parameters spaces and select points
- Random search:
 - Like grid search but one picks points at random in the predefined ranges



Simple Starting Points for Debugging

- Start simple!
 - First, overfit to a single training sample
 - Second, overfit to several training samples
- Always try simple architecture first
 - It will verify that you are learning something
- Estimate timings (how long for each epoch?)



Debugging

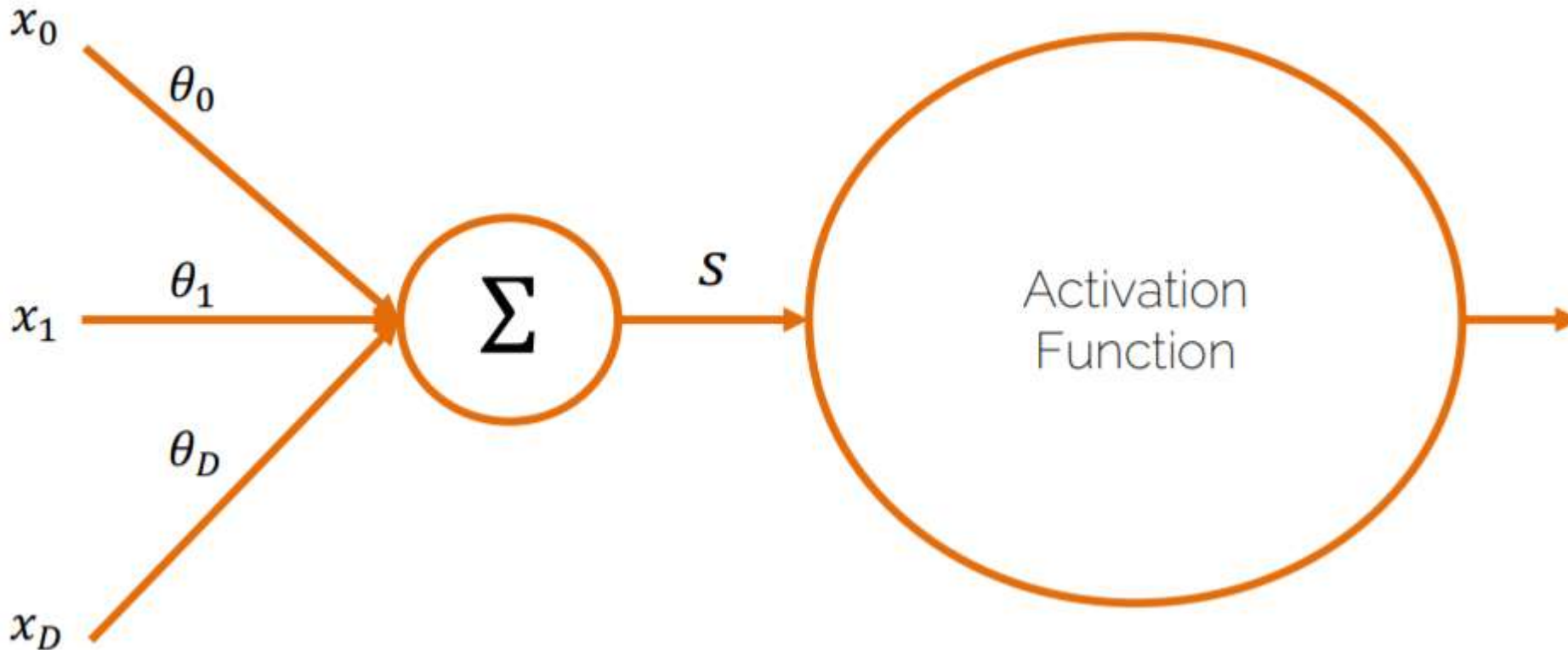
- Use train/validation/test curves
 - Evaluation needs to be consistent
 - Numbers need to be comparable
- Only make one change at a time
 - “I’ve added 5 more layers and double the training size, and now I also trained 5 days longer. Now it’s better, but why?”

Training NN

- Activation Functions
- Data Preprocessing
- Weight Initialization
- Batch Normalization

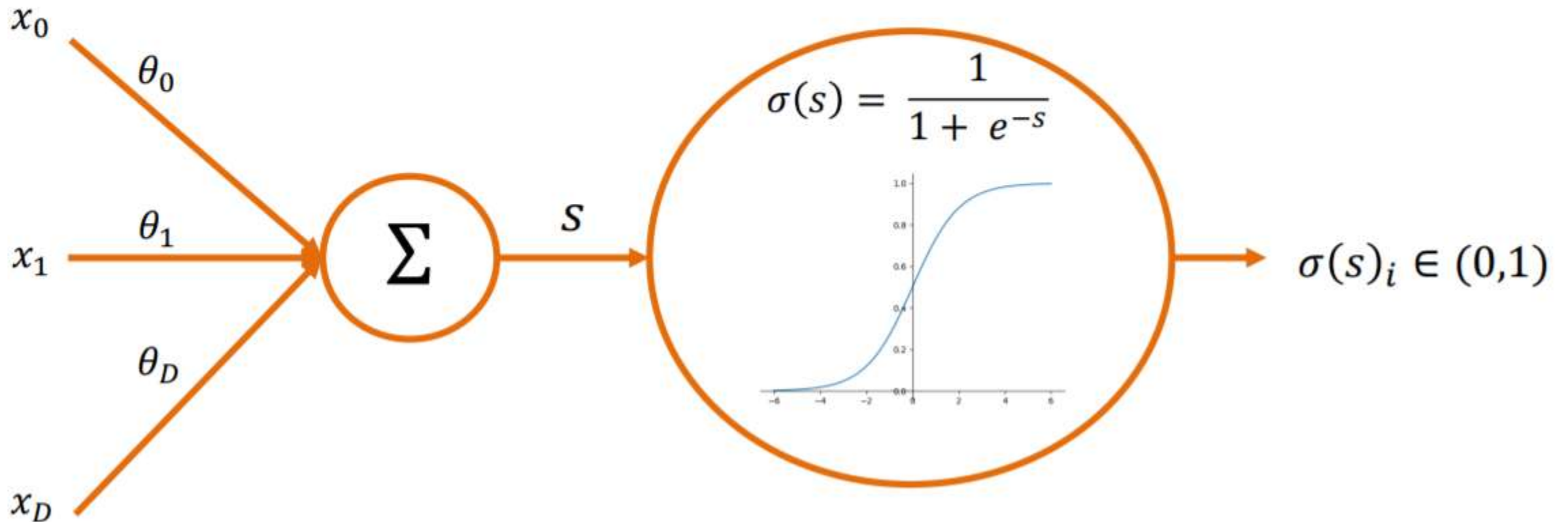
Activation Functions

Activation Functions or Hidden Units



Sigmoid Activation

$$\sigma(s) = \frac{1}{1 + e^{-s}}$$



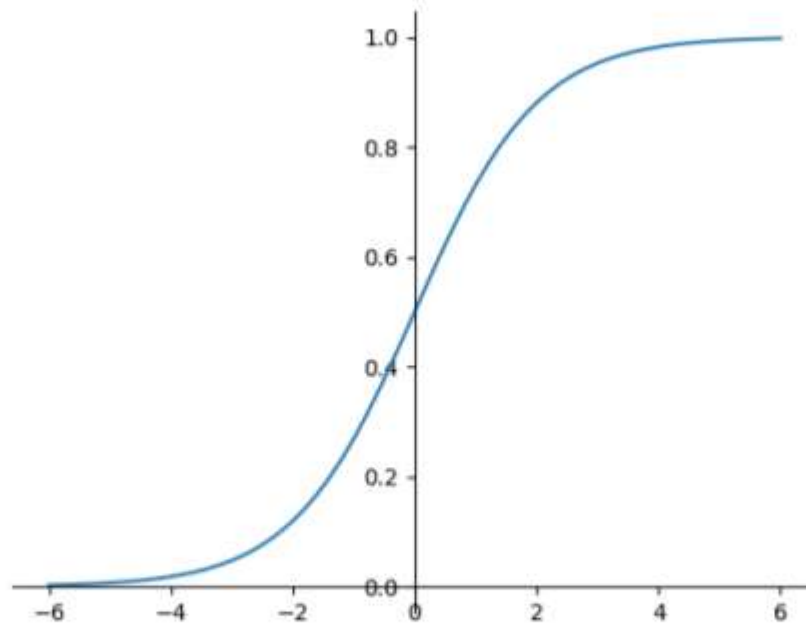
Sigmoid Activation

$$s = w_1x_1 + w_2x_2 + \dots + w_Dx_D + b$$

Forward



$$\sigma(s) = \frac{1}{1 + e^{-s}}$$



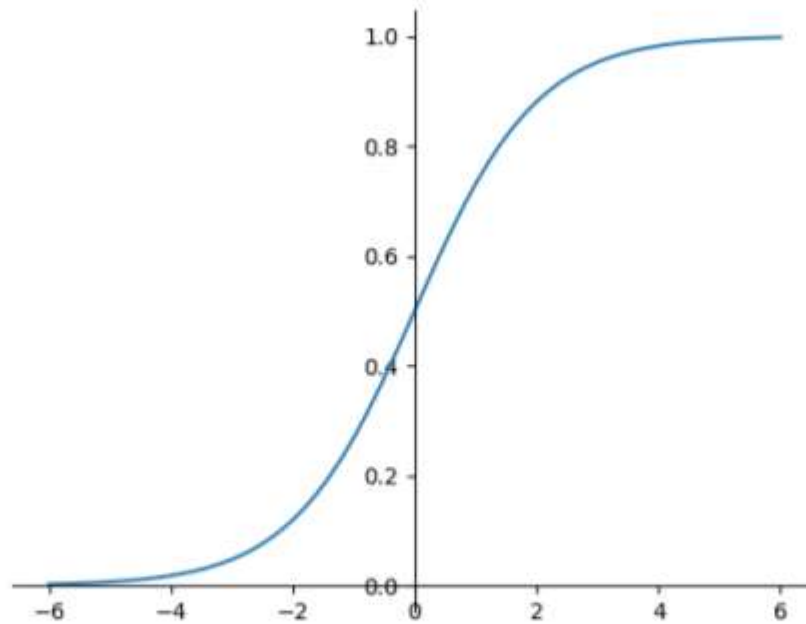
Sigmoid Activation

$$s = w_1x_1 + w_2x_2 + \dots + w_Dx_D + b$$

Forward



$$\sigma(s) = \frac{1}{1 + e^{-s}}$$



$$\frac{\partial L}{\partial s} = \frac{\partial \sigma}{\partial s} \frac{\partial L}{\partial \sigma}$$



$$\frac{\partial \sigma}{\partial s}$$



$$\frac{\partial L}{\partial \sigma}$$

Sigmoid Activation

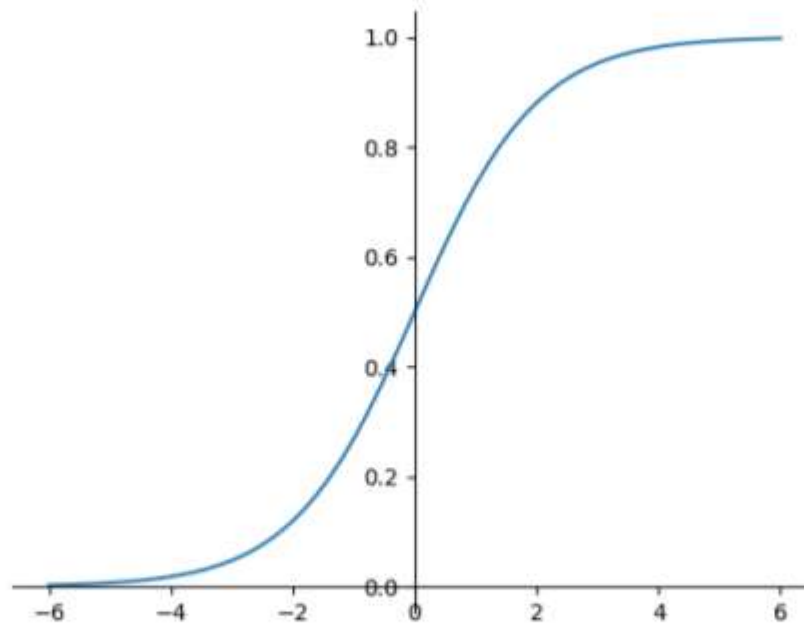
$$s = w_1x_1 + w_2x_2 + \dots + w_Dx_D + b$$

Forward



$$\sigma(s) = \frac{1}{1 + e^{-s}}$$

$$\frac{\partial L}{\partial w} = \frac{\partial s}{\partial w} \frac{\partial L}{\partial s}$$



$$\frac{\partial L}{\partial s} = \frac{\partial \sigma}{\partial s} \frac{\partial L}{\partial \sigma}$$



$$\frac{\partial \sigma}{\partial s}$$



$$\frac{\partial L}{\partial \sigma}$$

Sigmoid Activation

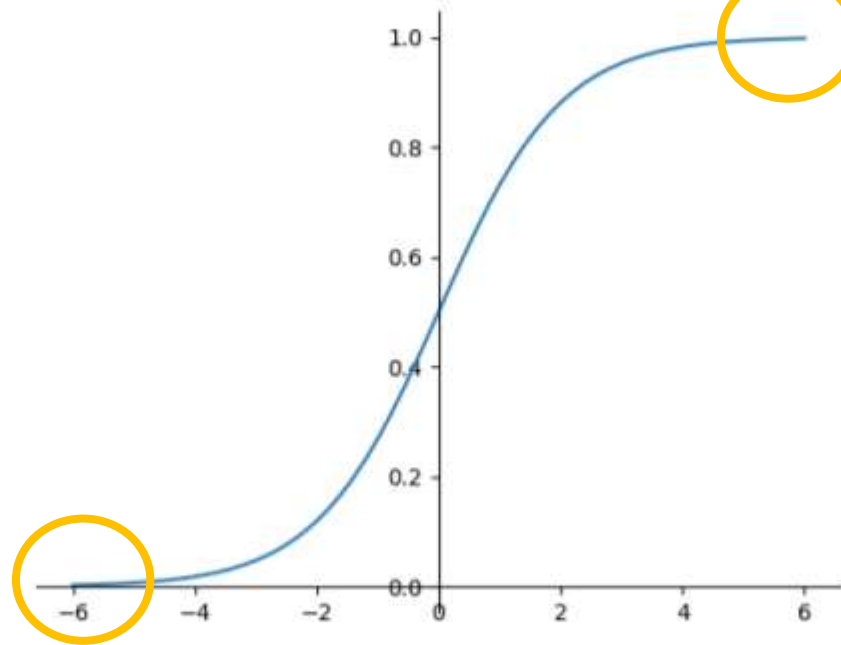
$$s = w_1x_1 + w_2x_2 + \dots + w_Dx_D + b$$

Forward



$$\sigma(s) = \frac{1}{1 + e^{-s}}$$

$$\frac{\partial L}{\partial w} = \frac{\partial s}{\partial w} \frac{\partial L}{\partial s}$$



$$\frac{\partial L}{\partial s} = \frac{\partial \sigma}{\partial s} \frac{\partial L}{\partial \sigma}$$



$$\frac{\partial \sigma}{\partial s}$$



$$\frac{\partial L}{\partial \sigma}$$

Sigmoid Activation

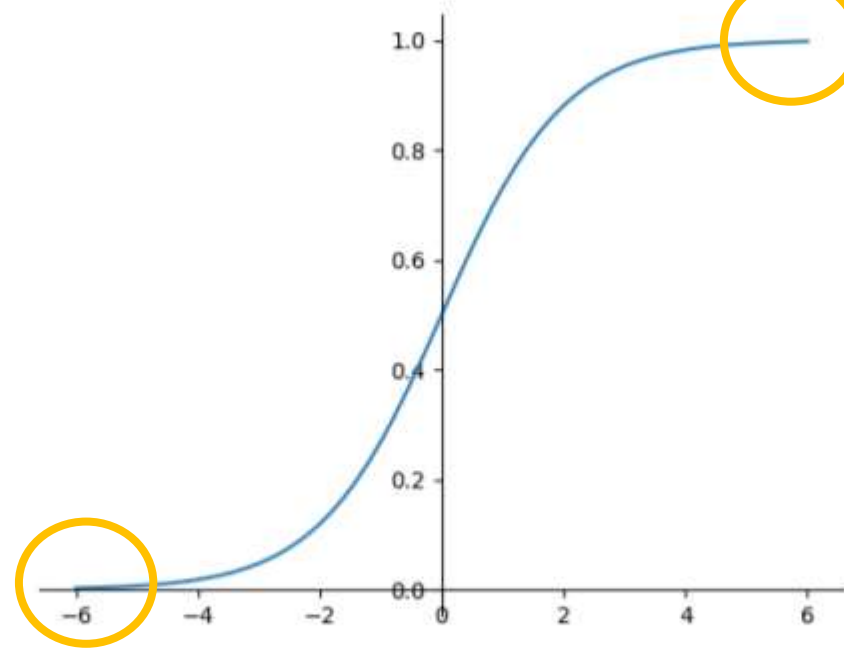
$$s = w_1x_1 + w_2x_2 + \dots + w_Dx_D + b$$

Forward



$$\sigma(s) = \frac{1}{1 + e^{-s}}$$

$$\frac{\partial L}{\partial w} = \frac{\partial s}{\partial w} \frac{\partial L}{\partial s}$$



✗ Saturated neurons
kill the gradient flow

$$\frac{\partial L}{\partial s} = \frac{\partial \sigma}{\partial s} \frac{\partial L}{\partial \sigma}$$



$$\frac{\partial \sigma}{\partial s}$$



$$\frac{\partial L}{\partial \sigma}$$

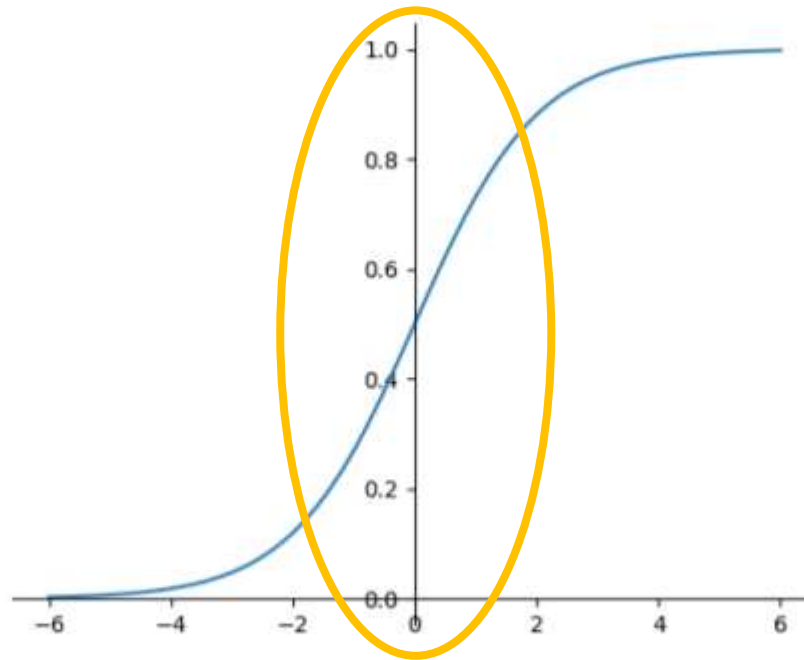
Sigmoid Activation

Forward



$$\sigma(s) = \frac{1}{1 + e^{-s}}$$

$$\frac{\partial L}{\partial w} = \frac{\partial s}{\partial w} \frac{\partial L}{\partial s}$$



Active region for
gradient descent

$$\frac{\partial L}{\partial s} = \frac{\partial \sigma}{\partial s} \frac{\partial L}{\partial \sigma}$$



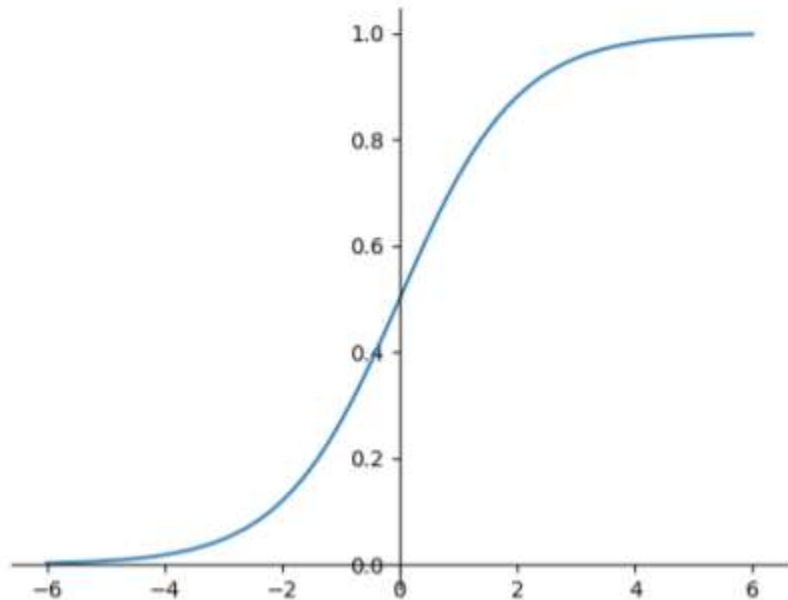
$$\frac{\partial \sigma}{\partial s}$$



$$\frac{\partial L}{\partial \sigma}$$

Sigmoid Activation

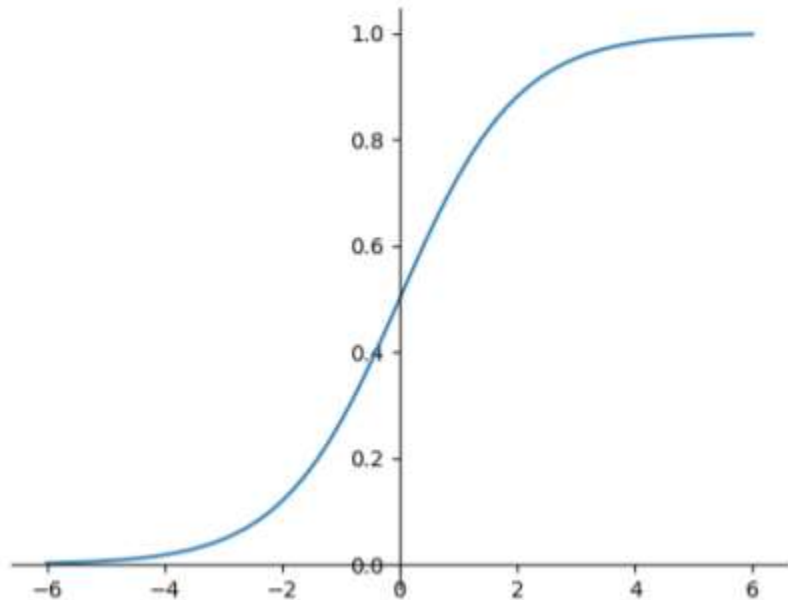
$$\sigma(s) = \frac{1}{1 + e^{-s}}$$



Sigmoid Activation

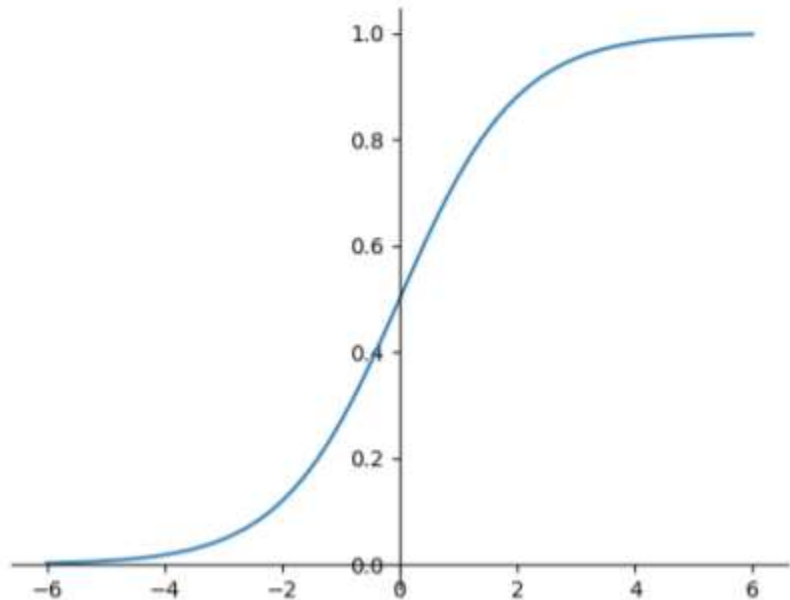
$$\sigma(s) = \frac{1}{1 + e^{-s}}$$

Output is always positive!



Sigmoid Activation

$$\sigma(s) = \frac{1}{1 + e^{-s}}$$

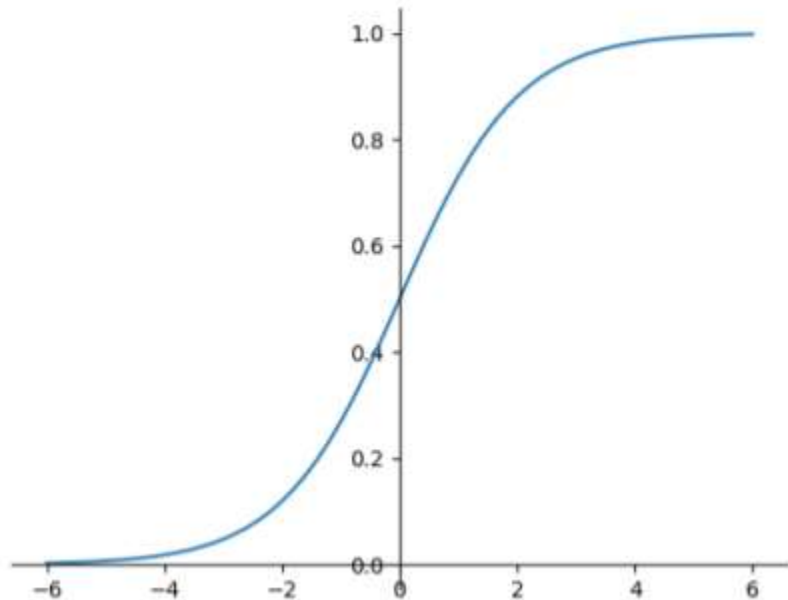


Output is always positive!

Sigmoid output provides positive input for the next layer

Sigmoid Activation

$$\sigma(s) = \frac{1}{1 + e^{-s}}$$



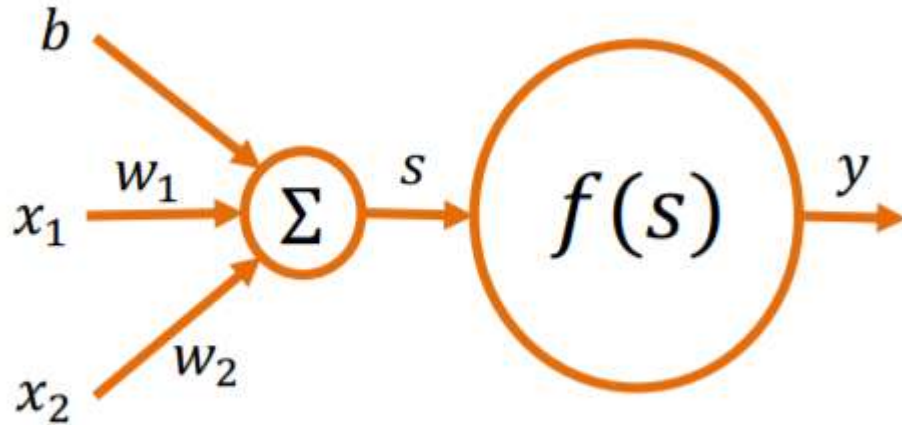
Output is always positive!

Sigmoid output provides positive input for the next layer

What is the disadvantage of this?

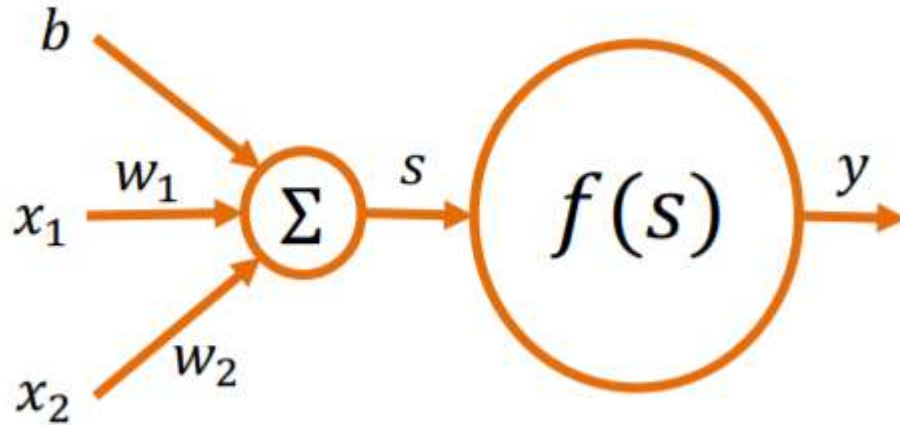
Sigmoid Output not Zero-centered

We want to compute the gradient w.r.t. the weights



Sigmoid Output not Zero-centered

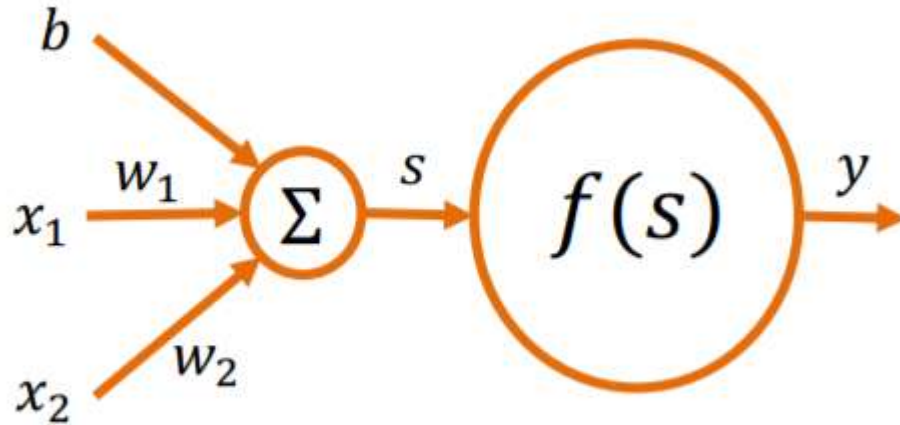
We want to compute the gradient w.r.t. the weights



$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial s} \cdot \frac{\partial s}{\partial w_1}$$
$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial s} \cdot \frac{\partial s}{\partial w_2}$$

Sigmoid Output not Zero-centered

We want to compute the gradient w.r.t. the weights

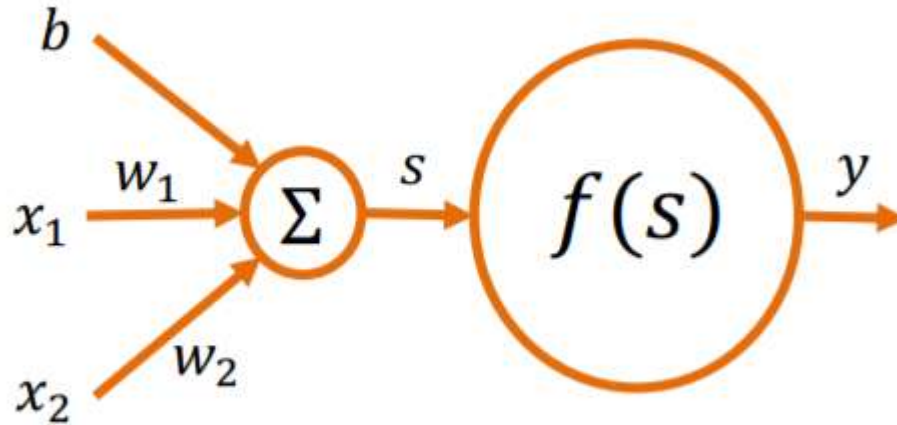


$$\begin{aligned}\frac{\partial L}{\partial w_1} &= \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial s} \cdot \frac{\partial s}{\partial w_1} \\ \frac{\partial L}{\partial w_2} &= \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial s} \cdot \frac{\partial s}{\partial w_2}\end{aligned}$$

The diagram shows the chain rule for the gradients of the loss L with respect to the weights w_1 and w_2 . The first two terms, $\frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial s}$, are grouped in an orange box. The third term, $\frac{\partial s}{\partial w_i}$, is shown in a blue oval. For $\frac{\partial s}{\partial w_1}$, a blue arrow labeled x_1 points to the $\frac{\partial s}{\partial w_1}$ term. For $\frac{\partial s}{\partial w_2}$, a blue arrow labeled x_2 points to the $\frac{\partial s}{\partial w_2}$ term.

Sigmoid Output not Zero-centered

We want to compute the gradient w.r.t. the weights



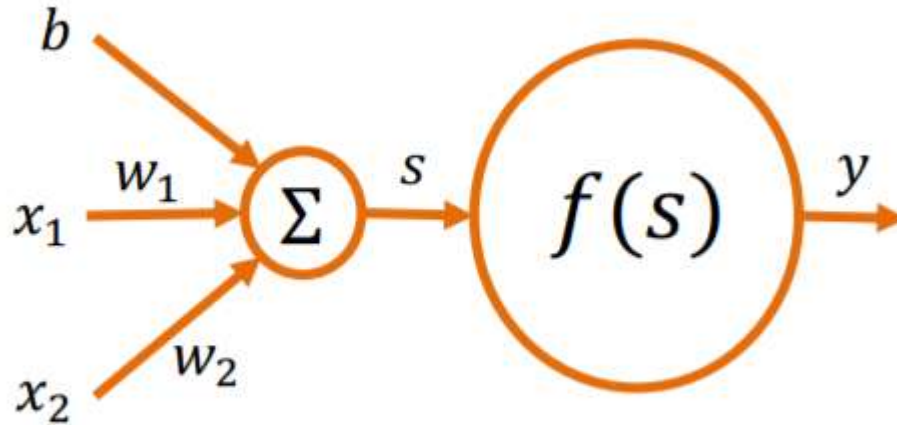
either positive
or negative

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial s} \cdot \frac{\partial s}{\partial w_1}$$
$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial s} \cdot \frac{\partial s}{\partial w_2}$$

The diagram shows the chain rule for computing gradients. The first two terms, $\frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial s}$, are grouped in an orange box. An orange arrow points from the text "either positive or negative" to this box. The third term, $\frac{\partial s}{\partial w_i}$, is shown in a blue oval. For the first equation, a blue arrow labeled x_1 points to the $\frac{\partial s}{\partial w_1}$ term. For the second equation, a blue arrow labeled x_2 points to the $\frac{\partial s}{\partial w_2}$ term.

Sigmoid Output not Zero-centered

We want to compute the gradient w.r.t. the weights



Assume we have all positive data:
 $\mathbf{x} = (x_1, x_2)^T > 0$

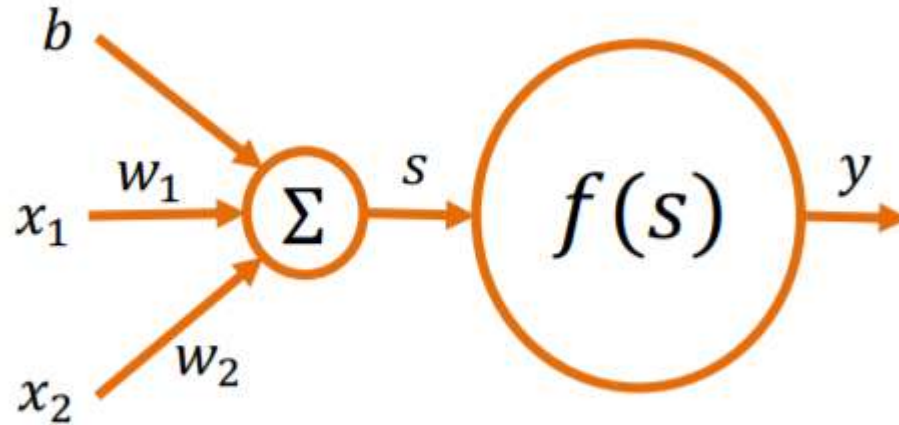
either positive
or negative

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial s} \cdot \frac{\partial s}{\partial w_1}$$
$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial s} \cdot \frac{\partial s}{\partial w_2}$$

The diagram highlights the chain rule for the gradients of the loss L with respect to the weights w_1 and w_2 . The first two terms, $\frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial s}$, are grouped in an orange box and labeled "either positive or negative". The third term, $\frac{\partial s}{\partial w_i}$, is shown in a blue oval. For w_1 , the derivative $\frac{\partial s}{\partial w_1}$ is multiplied by x_1 (indicated by a blue arrow). For w_2 , the derivative $\frac{\partial s}{\partial w_2}$ is multiplied by x_2 (indicated by a blue arrow).

Sigmoid Output not Zero-centered

We want to compute the gradient w.r.t. the weights



Assume we have all positive data:
 $\mathbf{x} = (x_1, x_2)^T > 0$

either positive
or negative

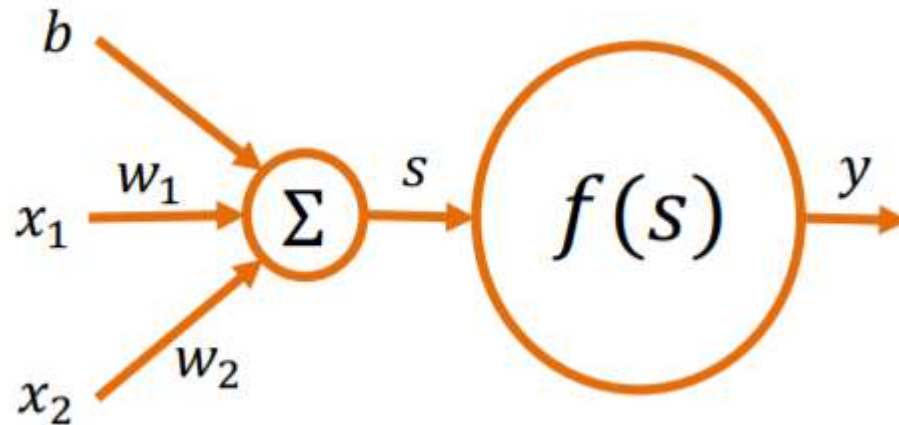
$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial s} \cdot \frac{\partial s}{\partial w_1}$$
$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial s} \cdot \frac{\partial s}{\partial w_2}$$

$x_1 > 0$

$x_2 > 0$

Sigmoid Output not Zero-centered

We want to compute the gradient w.r.t. the weights



Assume we have all positive data:

$$\mathbf{x} = (x_1, x_2)^T > 0$$

either positive
or negative

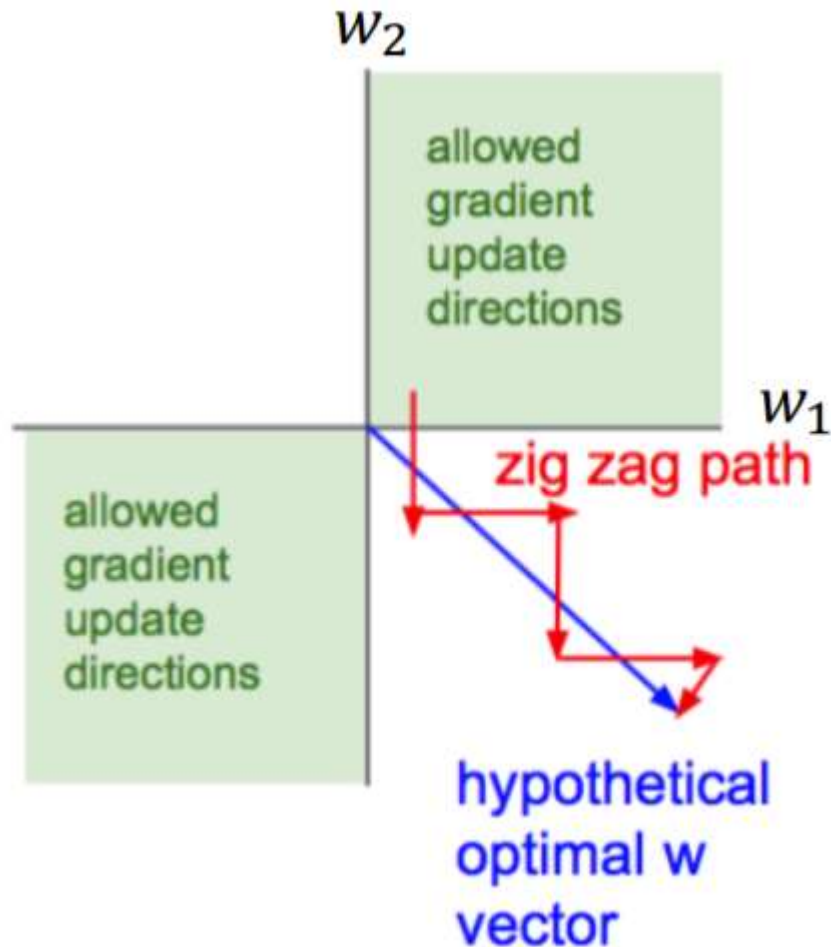
$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial s} \cdot \frac{\partial s}{\partial w_1}$$
$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial s} \cdot \frac{\partial s}{\partial w_2}$$

$x_1 > 0$

$x_2 > 0$

It is going to be either positive or negative for all weights' update.

Sigmoid Output not Zero-centered

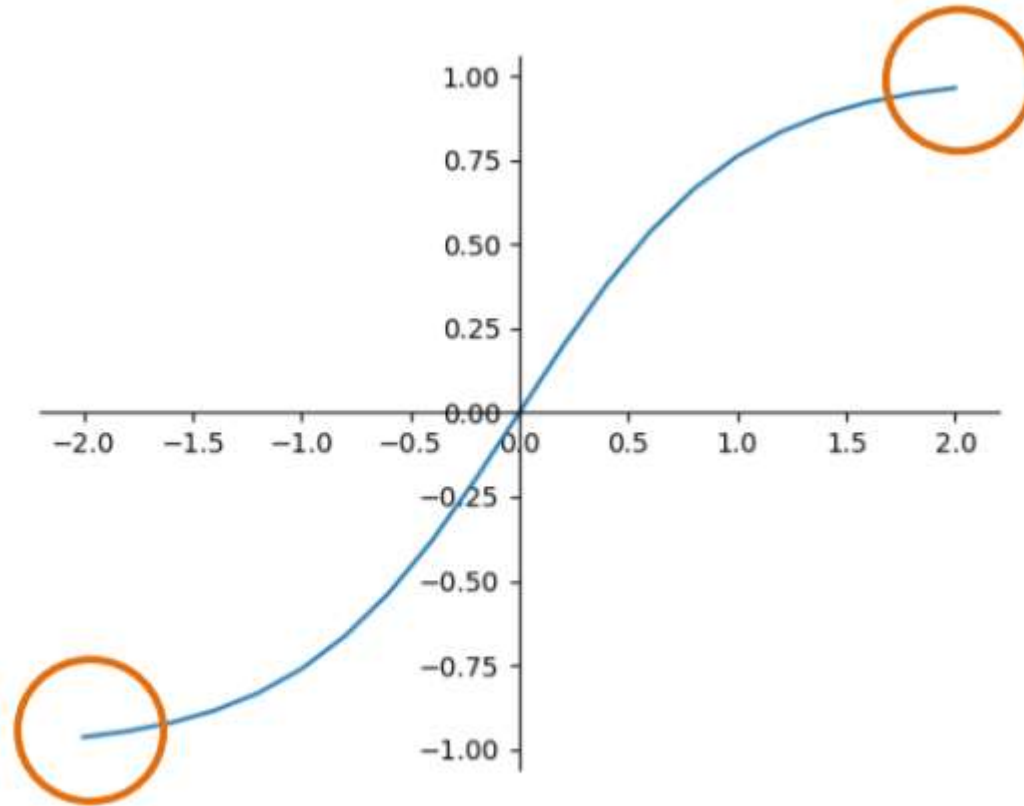


w_1 , w_2 can only be increased or decreased at the same time, which is not good for update.

That is also why you need zero-centered data.

Source : http://cs231n.stanford.edu/slides/2017/cs231n_2017_lecture6.pdf

TanH Activation



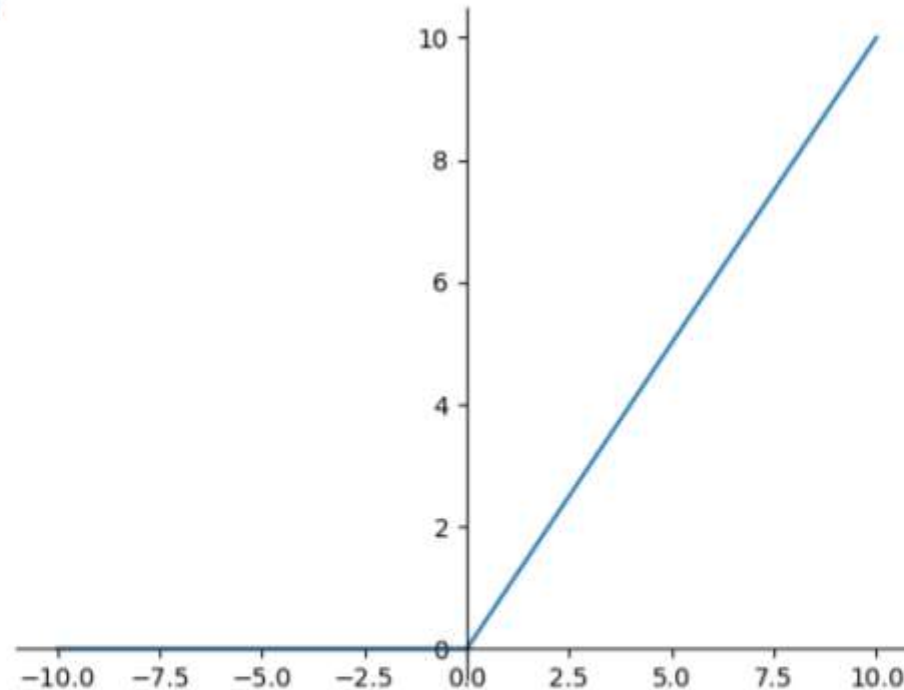
✗ Still saturates

✓ Zero-centered

[LeCun et al. 1991] Improving Generalization Performance in Character Recognition

Rectified Linear Units (ReLU)

$$\sigma(x) = \max(0, x)$$



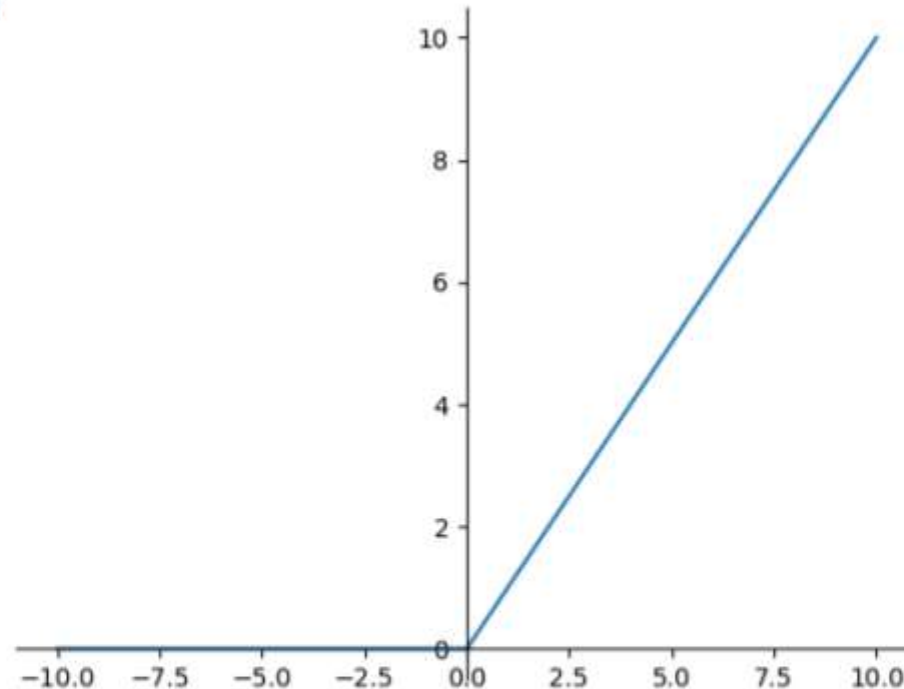
✓ Fast convergence

✓ Does not saturate

[Krizhevsky et al. NeurIPS 2012] ImageNet Classification with Deep Convolutional Neural Networks

Rectified Linear Units (ReLU)

$$\sigma(x) = \max(0, x)$$



Large and
consistent
gradients ✓

✓ Fast convergence

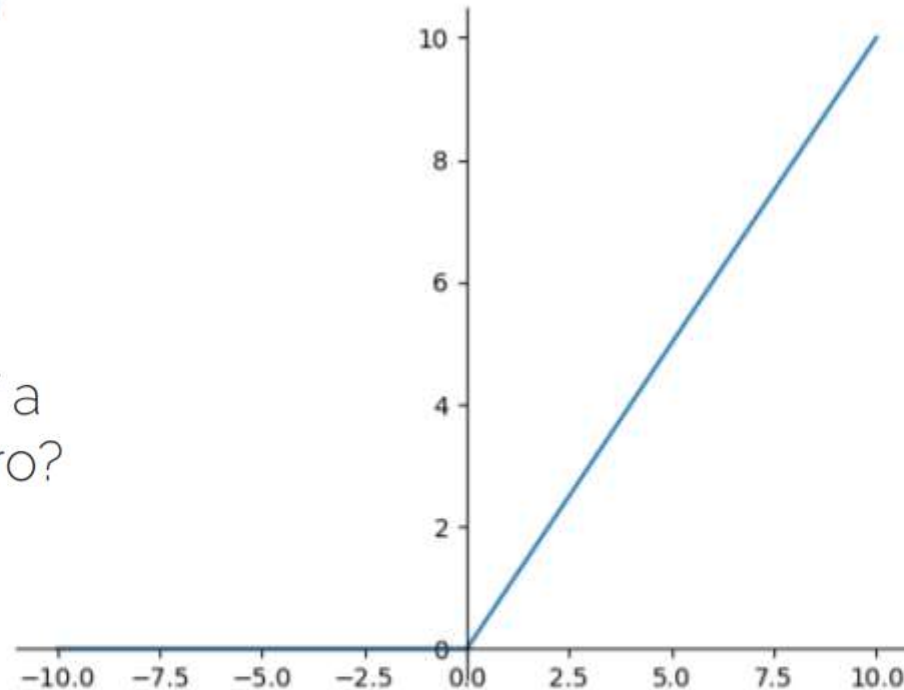
✓ Does not saturate

[Krizhevsky et al. NeurIPS 2012] ImageNet Classification with Deep Convolutional Neural Networks

Rectified Linear Units (ReLU)

$$\sigma(x) = \max(0, x)$$

What happens if a ReLU outputs zero?



Large and consistent gradients ✓

✓ Fast convergence

✓ Does not saturate

[Krizhevsky et al. NeurIPS 2012] ImageNet Classification with Deep Convolutional Neural Networks

Rectified Linear Units (ReLU)

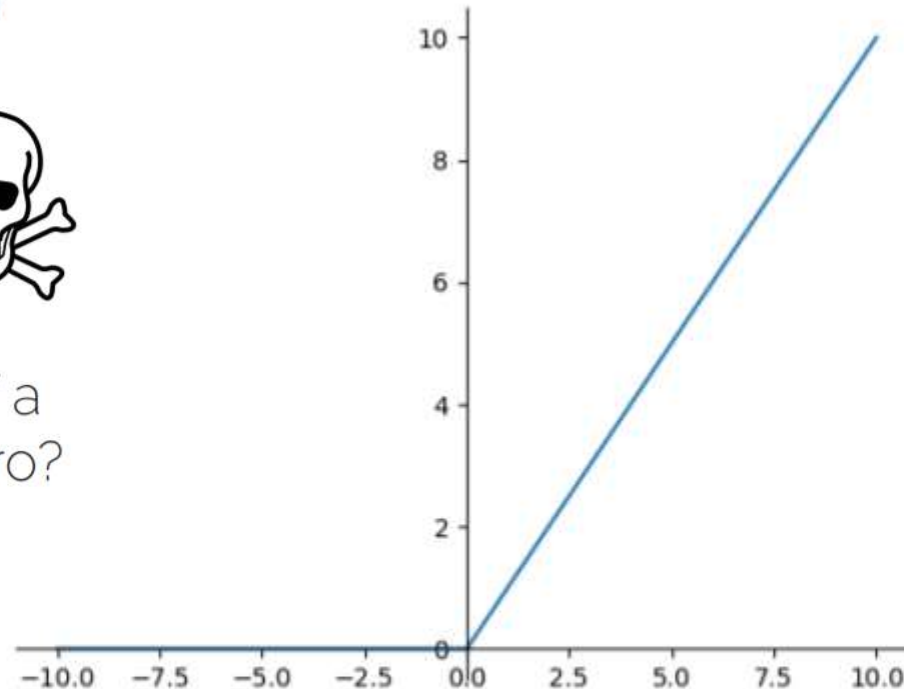
$$\sigma(x) = \max(0, x)$$



Dead ReLU



What happens if a ReLU outputs zero?



Large and consistent gradients



Fast convergence



Does not saturate

[Krizhevsky et al. NeurIPS 2012] ImageNet Classification with Deep Convolutional Neural Networks

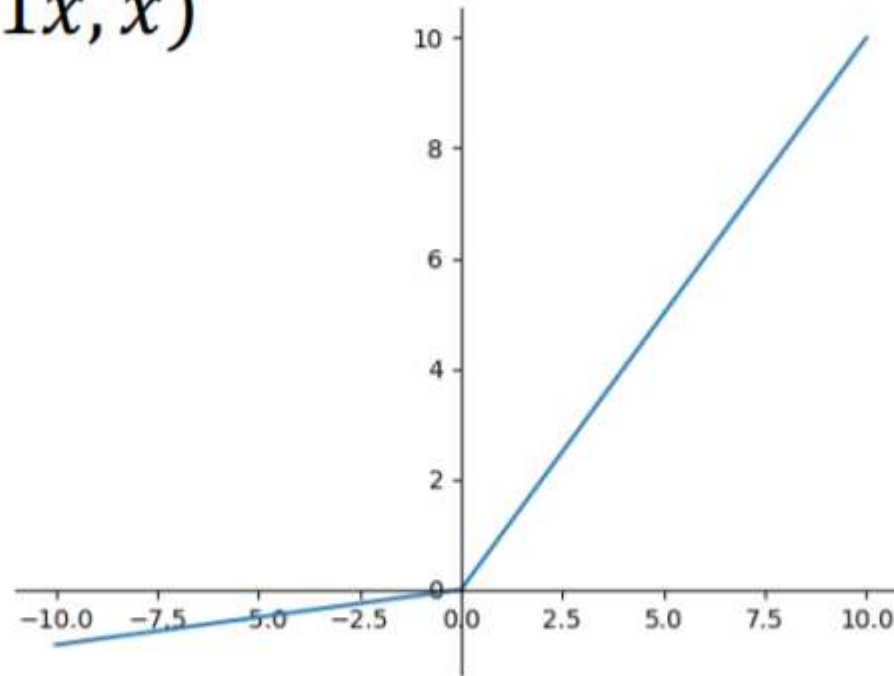
Rectified Linear Units (ReLU)

- Initializing ReLU neurons with slightly positive biases (0.01) makes it likely that they stay active for most inputs

$$f\left(\sum_i w_i x_i + b\right)$$

Leaky ReLU

$$\sigma(x) = \max(0.01x, x)$$



Does not die

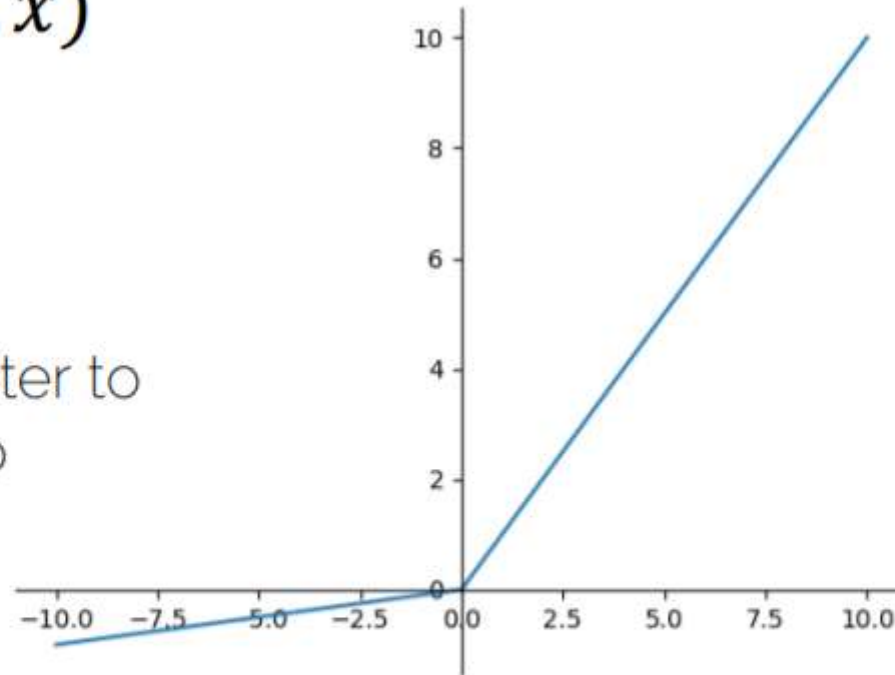
[Mass et al., ICML 2013] Rectifier Nonlinearities Improve Neural Network Acoustic Models

Parametric ReLU

$$\sigma(x) = \max(\alpha x, x)$$



One more parameter to
backprop into

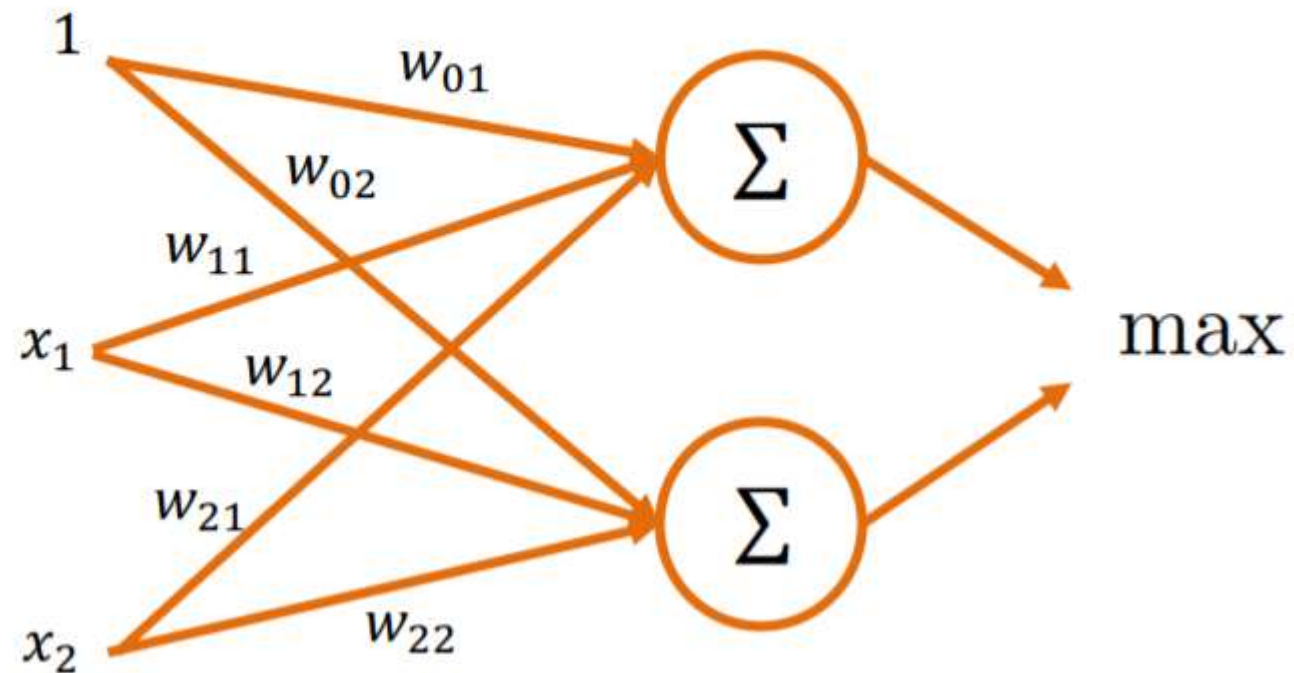


Does not die

[He et al. ICCV 2015] Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification

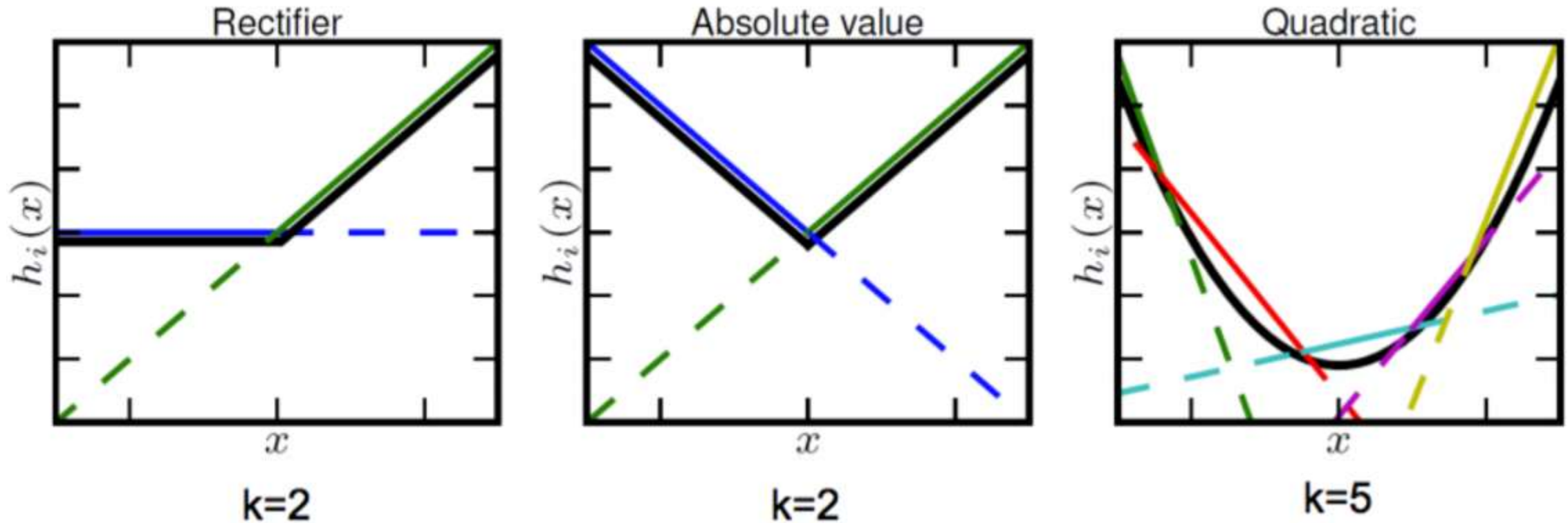
Maxout Units

$$\text{Maxout} = \max(w_1^T x + b_1, w_2^T x + b_2)$$



[Goodfellow et al. ICML 2013]] Maxout Networks

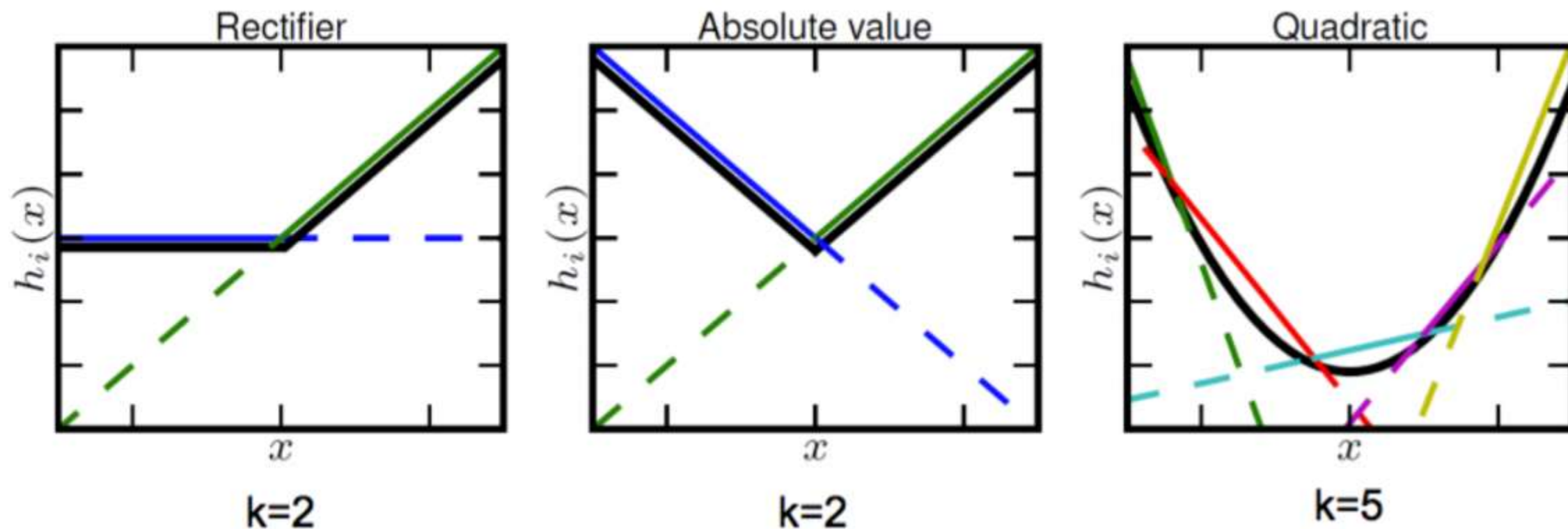
Maxout Units



Piecewise linear approximation of a convex function with N pieces

[Goodfellow et al. ICML 2013]] Maxout Networks

Maxout Units



✓ Generalization
of ReLUs

✓ Linear
regimes

✓ Does not
die

✓ Does not
saturate

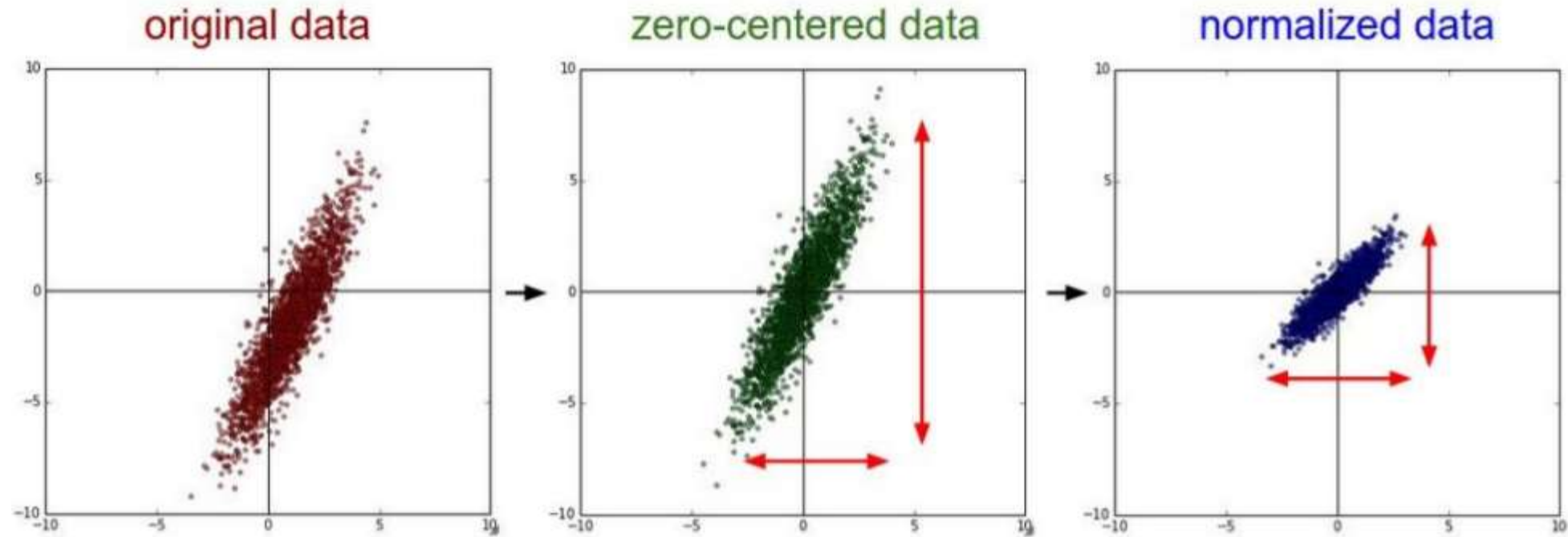
✗ Increases of the number of parameters

Quick Guide

- Sigmoid is not really used.
- ReLU is the standard choice.
- Second choice are the variants of ReLU or Maxout.
- Recurrent nets will require TanH or similar.

Data Preprocessing

Data Preprocessing



Data Preprocessing

Consider what happens when the input to a neuron is always positive

$$f\left(\sum_i w_i x_i + b\right)$$

Data Preprocessing

Consider what happens when the input to a neuron is always positive

$$f\left(\sum_i w_i x_i + b\right)$$

What can we say about the gradients on w ?

Data Preprocessing

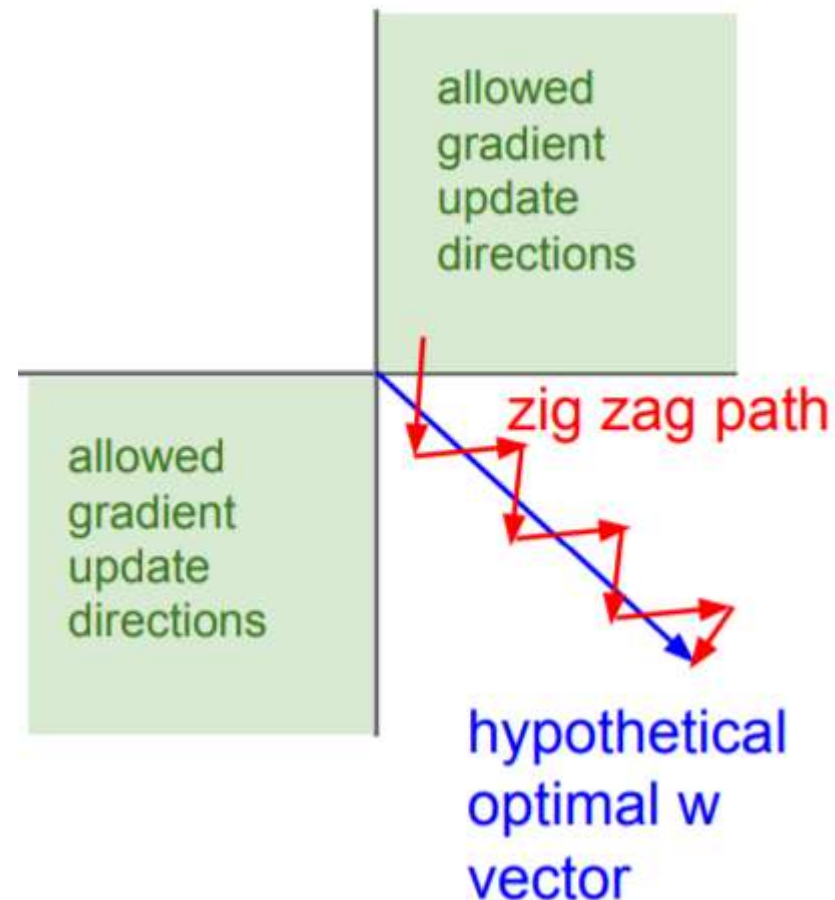
Consider what happens when the input to a neuron is always positive

$$f\left(\sum_i w_i x_i + b\right)$$

What can we say about the gradients on $\mathbf{w}$?

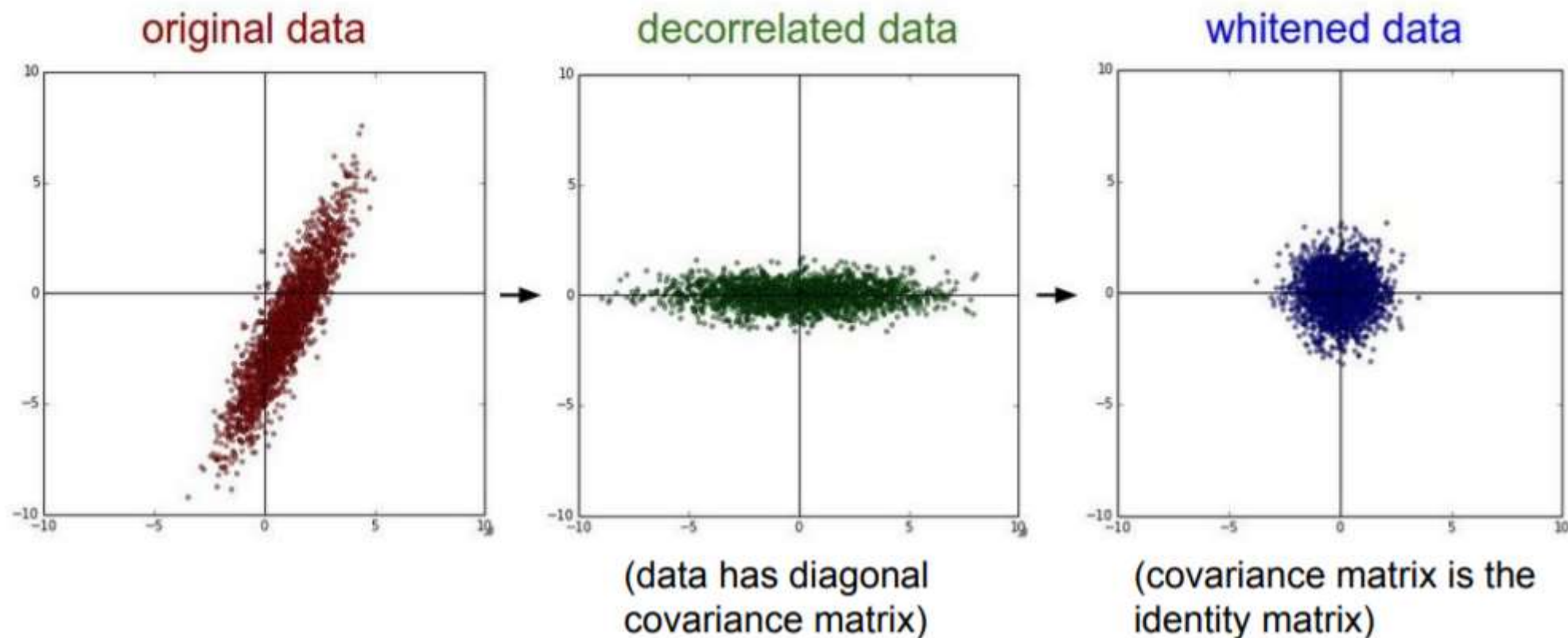
Always all positive or all negative :(

(this is also why you want zero-mean data!)



Data Preprocessing

In practice, you may also see **PCA** and **Whitening** of the data



In practice for Images

e.g. consider CIFAR-10 example with $[32,32,3]$ images

In practice for Images

e.g. consider CIFAR-10 example with $[32,32,3]$ images

- Subtract the mean image (e.g. AlexNet) (mean image = $[32,32,3]$ array)

In practice for Images

e.g. consider CIFAR-10 example with $[32,32,3]$ images

- Subtract the mean image (e.g. AlexNet) (mean image = $[32,32,3]$ array)
- Subtract per-channel mean (e.g. VGGNet) (mean along each channel = 3 numbers)

In practice for Images

e.g. consider CIFAR-10 example with $[32,32,3]$ images

- Subtract the mean image (e.g. AlexNet) (mean image = $[32,32,3]$ array)
- Subtract per-channel mean (e.g. VGGNet) (mean along each channel = 3 numbers)
- Subtract per-channel mean and divide by per-channel std (e.g. ResNet) (mean along each channel = 3 numbers)

In practice for Images

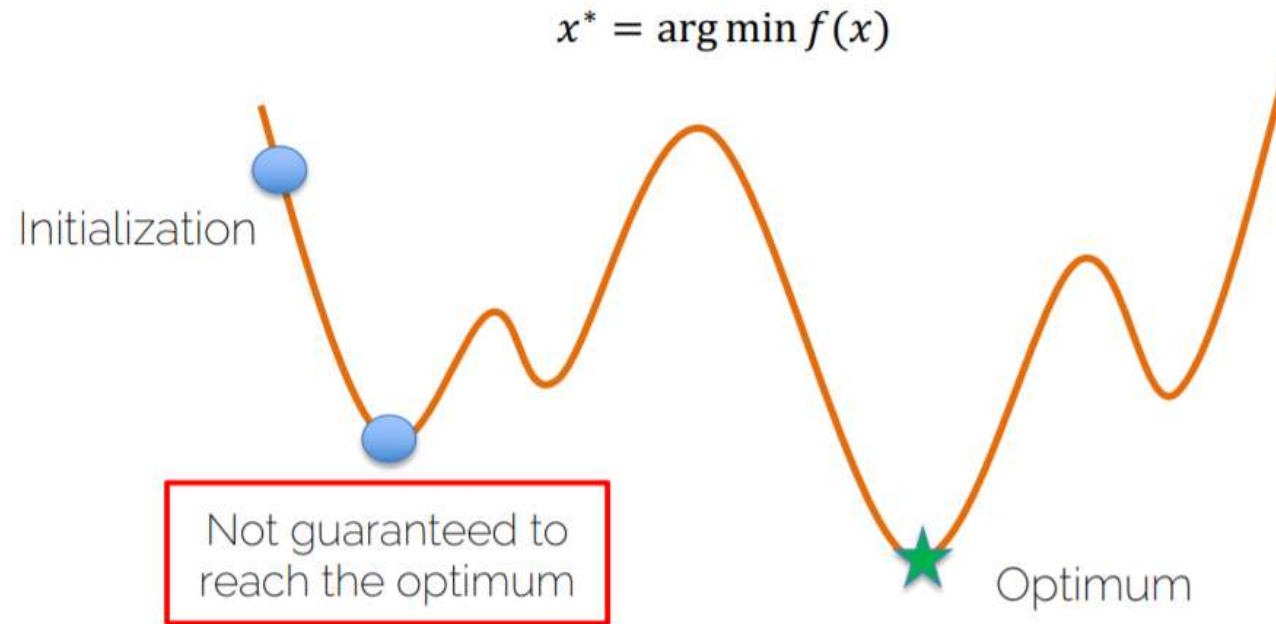
e.g. consider CIFAR-10 example with $[32,32,3]$ images

- Subtract the mean image (e.g. AlexNet) (mean image = $[32,32,3]$ array)
- Subtract per-channel mean (e.g. VGGNet) (mean along each channel = 3 numbers)
- Subtract per-channel mean and divide by per-channel std (e.g. ResNet) (mean along each channel = 3 numbers)

Not common
to do PCA or
whitening

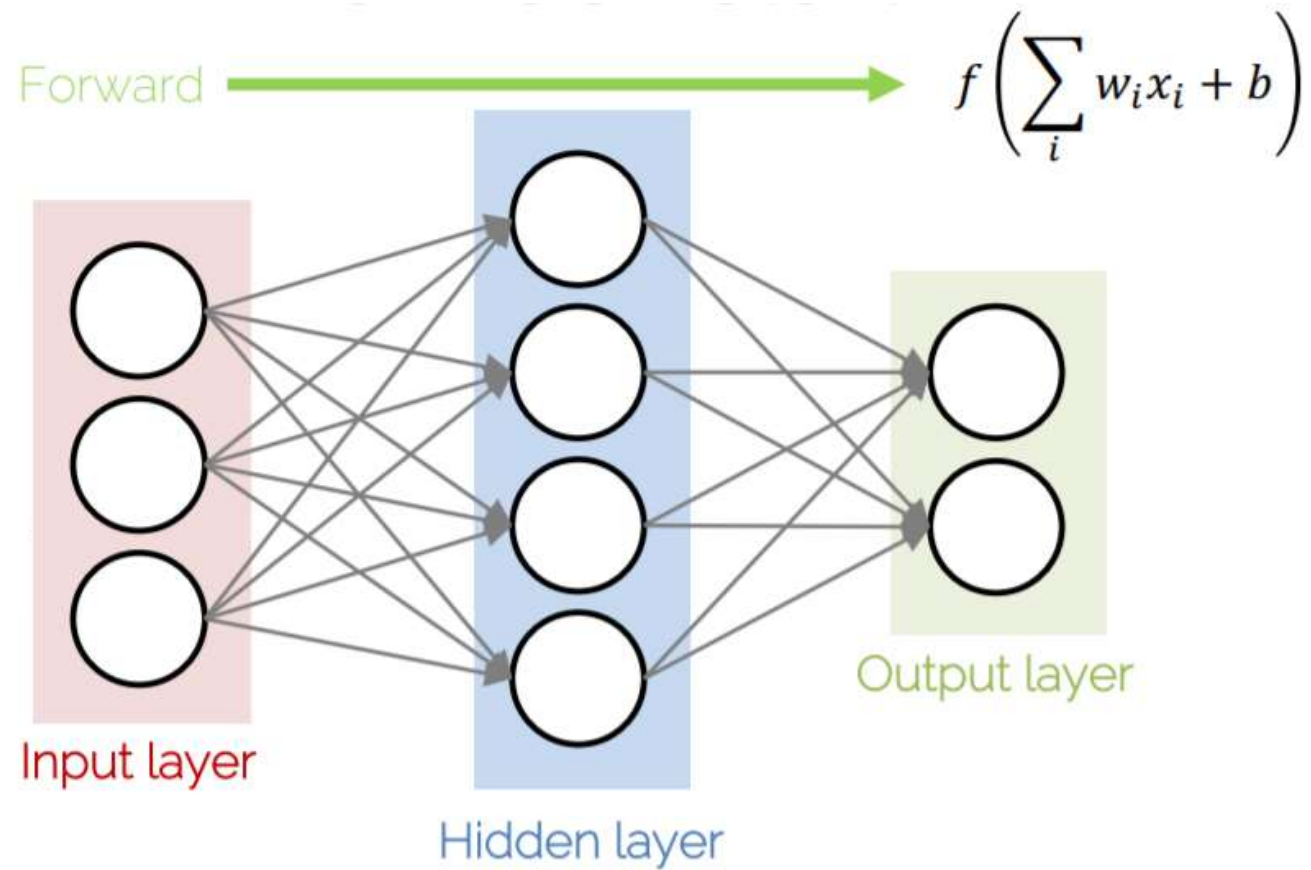
Weight Initialization

How do I start?

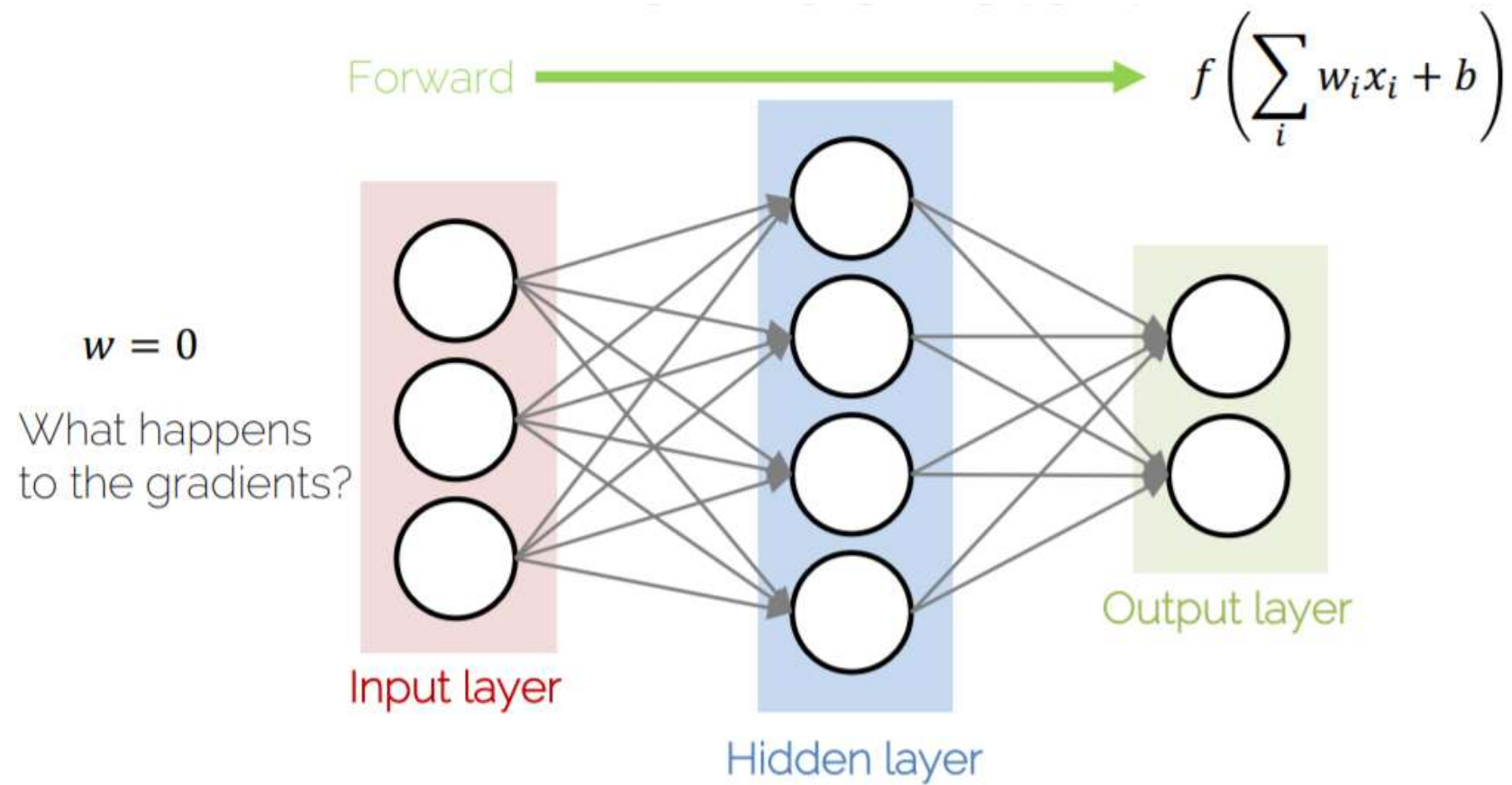


Initialization is extremely Important

How do I start?



How do I start?



All Weights Zero

- What happens to the gradients?

All Weights Zero

- What happens to the gradients?
- The hidden units are all going to compute the same function, gradients are going to be the same
 - No symmetry breaking

Small Random Numbers

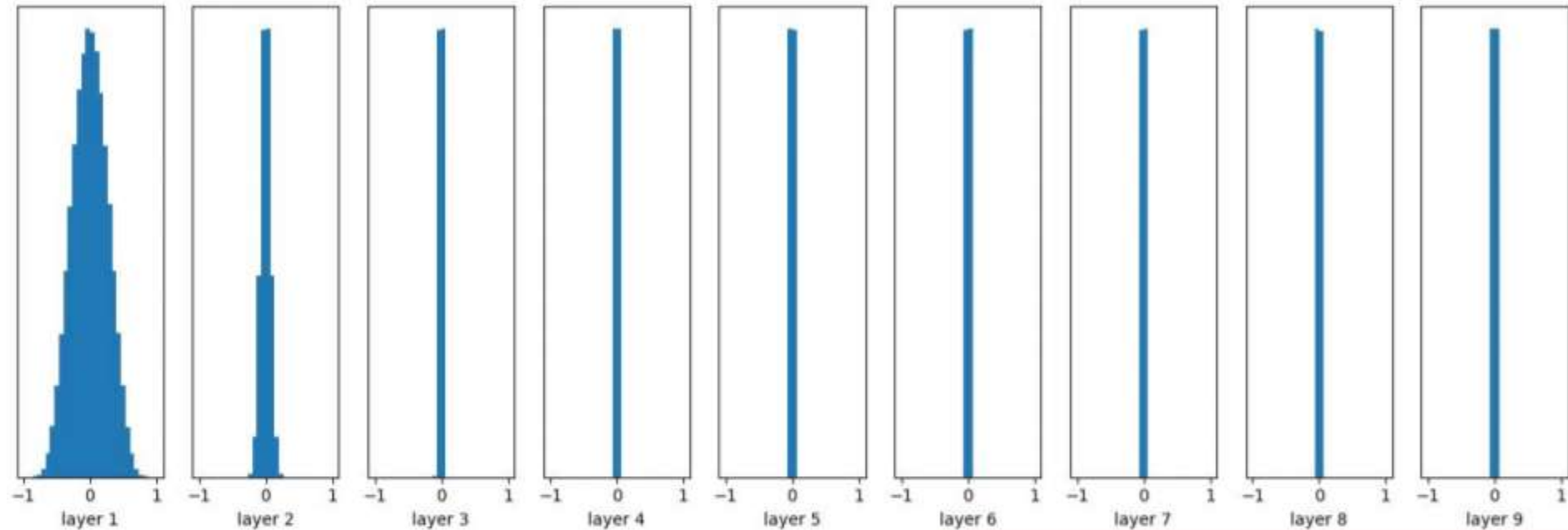
- Gaussian with zero mean and standard deviation 0.01

Small Random Numbers

- Gaussian with zero mean and standard deviation 0.01
- Let's see what happens:
 - Network with 10 layers with 500 neurons each
 - Tanh as activation functions
 - Input unit Gaussian data

Small Random Numbers

tanh as activation functions

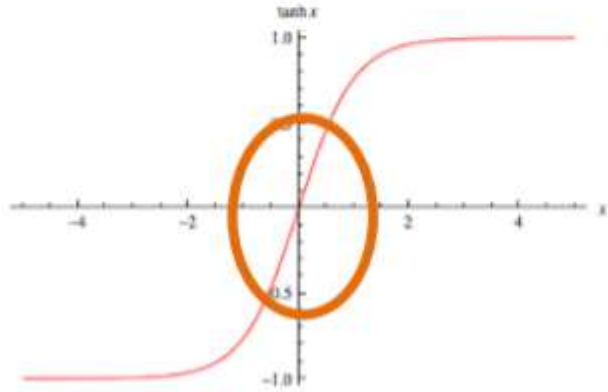


Output become to zero

Forward



Small Random Numbers



Small w_i^l cause small output for layer l :

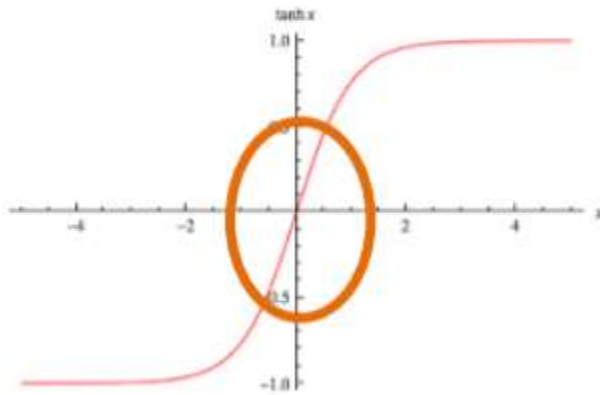
$$f_l \left(\sum_i w_i^l x_i^l + b^l \right) \approx 0$$

Forward



Small Random Numbers

Even activation function's gradient is ok, we still have vanishing gradient problem.

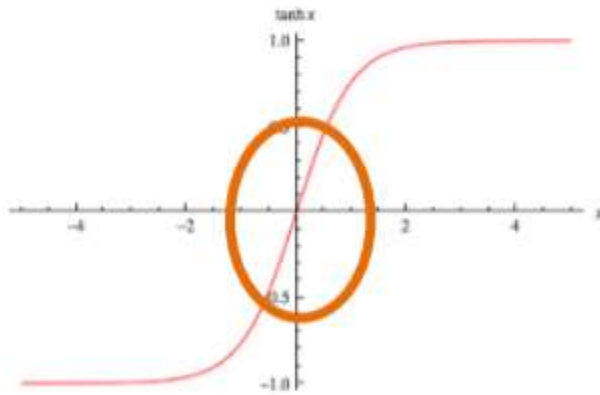


Backward



Small Random Numbers

Even activation function's gradient is ok, we still have vanishing gradient problem.



Small outputs of layer l (input of layer $l + 1$) cause small gradient w.r.t to the weights of layer $l + 1$:

$$f_{l+1} \left(\sum_i w_i^{l+1} x_i^{l+1} + b^{l+1} \right)$$

Backward

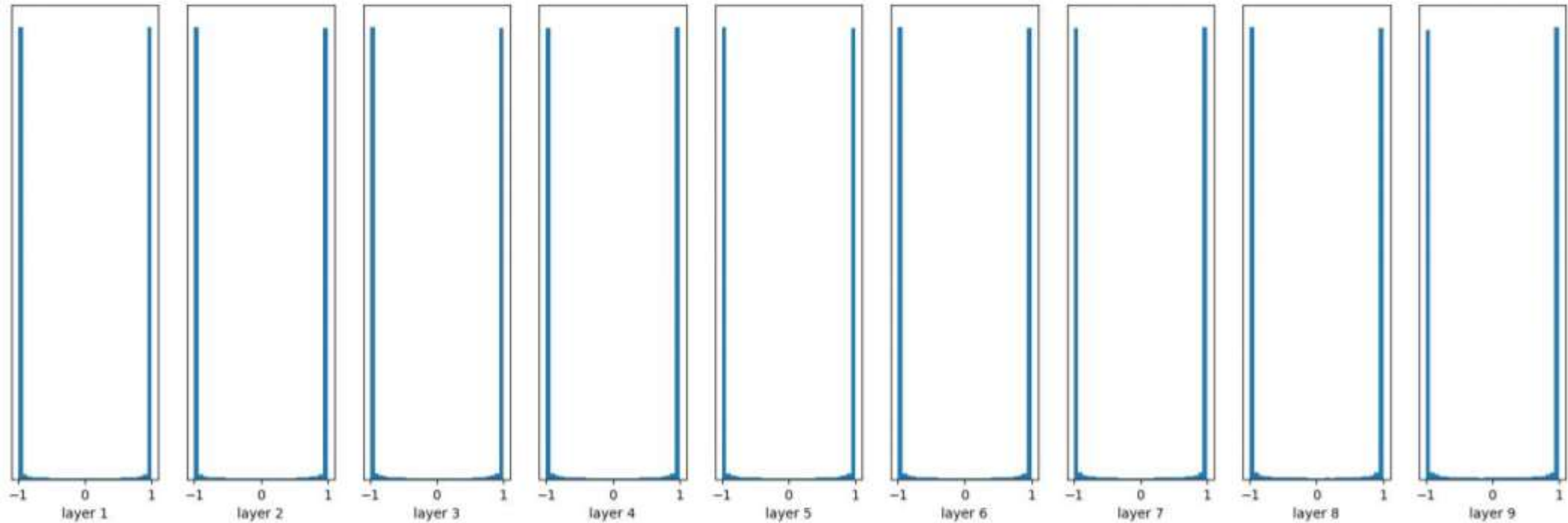


Big Random Numbers

- Gaussian with zero mean and standard deviation 1
- Let's see what happens:
 - Network with 10 layers with 500 neurons each
 - Tanh as activation functions
 - Input unit Gaussian data

Big Random Numbers

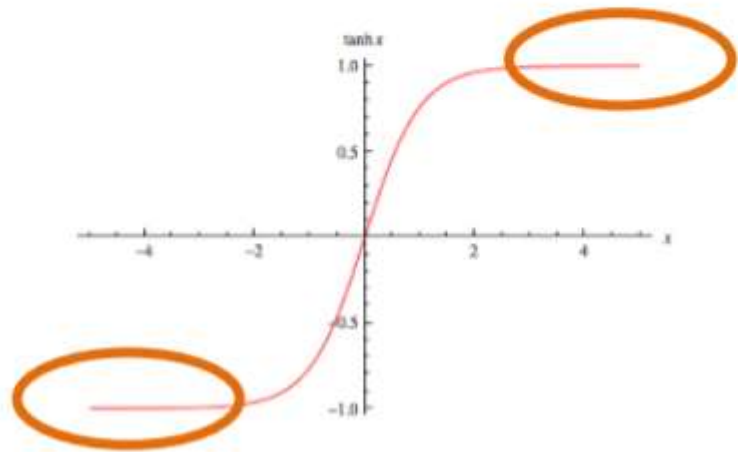
tanh as activation functions



Output saturated to
-1 and 1

Big Random Numbers

Output saturated to -1 and 1.
Gradient of the activation
function becomes close to 0.



$$f(s) = f\left(\sum_i w_i x_i + b\right)$$

$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial f} \cdot \frac{\partial f}{\partial s} \cdot \frac{\partial s}{\partial w_i} \approx 0$$

Vanishing gradient, caused by
saturated activation function.

How to solve this?

Xavier Initialization

Gaussian with zero mean, but what standard deviation?

How to solve this?

Xavier Initialization

Gaussian with zero mean, but what standard deviation?

$$\text{Var}(s) = \text{Var}\left(\sum_i^n w_i x_i\right) = \sum_i^n \text{Var}(w_i x_i)$$

n is the number of input neurons for the layer of weights you want to initialize.

For the first layer **n = D**.

Xavier Initialization

$$\begin{aligned} \text{Var}(s) &= \text{Var}\left(\sum_i^n w_i x_i\right) = \sum_i^n \text{Var}(w_i x_i) \quad \text{Independent} \\ &= \sum_i^n [E(w_i)]^2 \text{Var}(x_i) + E(x_i)^2 \text{Var}(w_i) + \text{Var}(x_i) \text{Var}(w_i) \end{aligned}$$

Zero mean Zero mean

Xavier Initialization

$$\begin{aligned} \text{Var}(s) &= \text{Var}\left(\sum_i^n w_i x_i\right) = \sum_i^n \text{Var}(w_i x_i) \quad \text{Independent} \\ &= \sum_i^n [E(w_i)]^2 \text{Var}(x_i) + E(x_i)^2 \text{Var}(w_i) + \text{Var}(x_i) \text{Var}(w_i) \\ &\quad \text{Zero mean} \quad \text{Zero mean} \\ &= \sum_i^n \text{Var}(x_i) \text{Var}(w_i) = n(\text{Var}(w) \text{Var}(x)) \quad \text{Identically distributed (each random variable has the same distribution)} \end{aligned}$$

Xavier Initialization

How to ensure the variance of the output is the same as the input?

Xavier Initialization

How to ensure the variance of the output is the same as the input?

Goal:

$$\text{Var}(s) = \text{Var}(x)$$

Xavier Initialization

How to ensure the variance of the output is the same as the input?

Goal:

$$\text{Var}(s) = \text{Var}(x) \quad \longrightarrow \quad n \cdot \text{Var}(w) \text{Var}(x) = \text{Var}(x)$$

Xavier Initialization

How to ensure the variance of the output is the same as the input?

Goal:

$$\text{Var}(s) = \text{Var}(x) \quad \longrightarrow \quad \underbrace{n \cdot \text{Var}(w)}_{= 1} \text{Var}(x) = \text{Var}(x)$$

Xavier Initialization

How to ensure the variance of the output is the same as the input?

Goal:

$$\text{Var}(s) = \text{Var}(x) \quad \longrightarrow \quad \underbrace{n \cdot \text{Var}(w)}_{=1} \text{Var}(x) = \text{Var}(x)$$

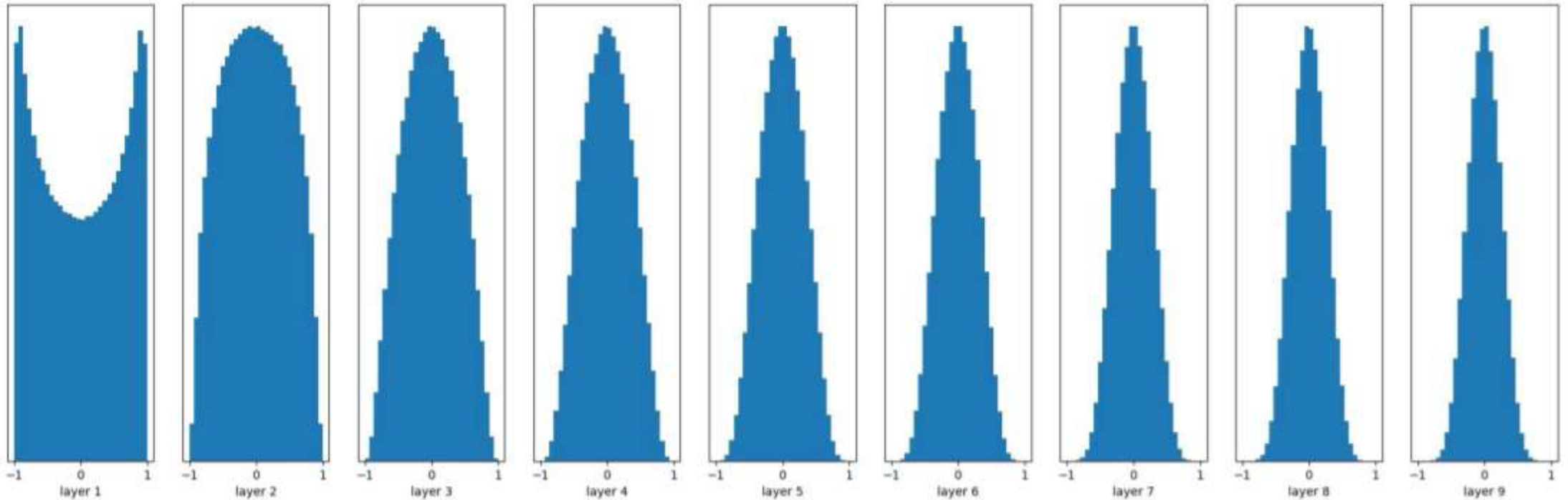
$$\longrightarrow \quad \text{Var}(w) = \frac{1}{n}$$

n : number of input neurons

Xavier Initialization

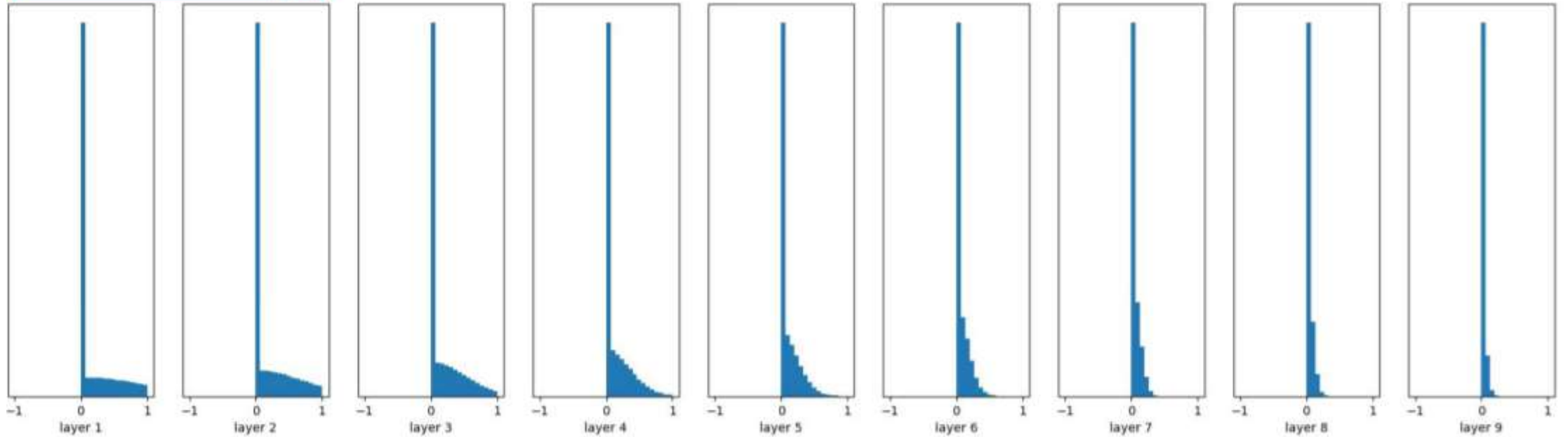
$$\text{Var}(w) = \frac{1}{n}$$

tanh as activation functions



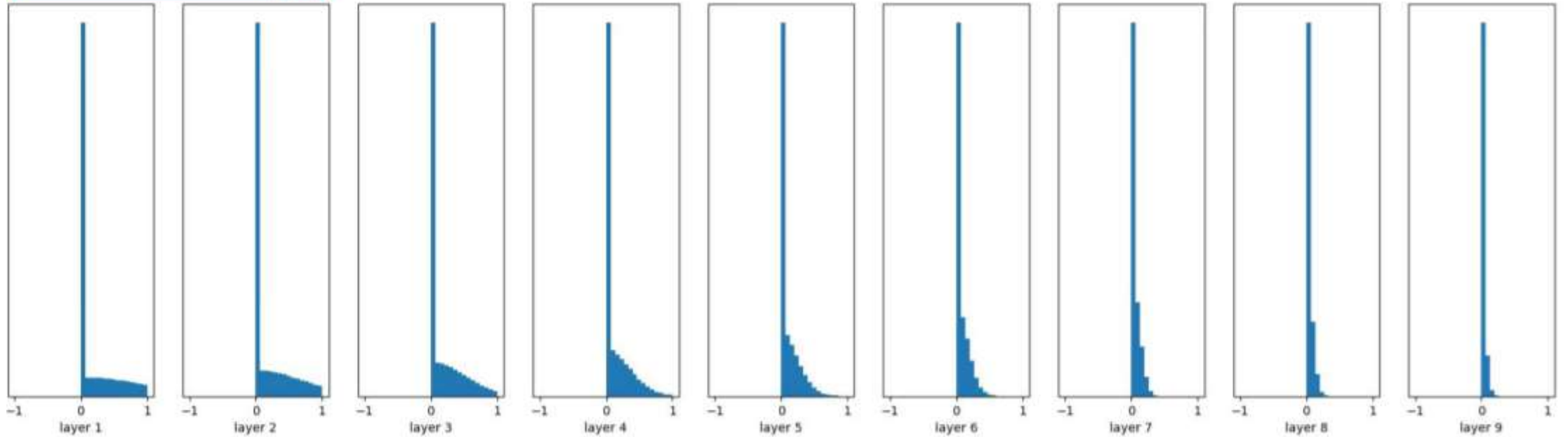
Xavier Initialization with ReLU

$$\text{Var}(w) = \frac{1}{n}$$



Xavier Initialization with ReLU

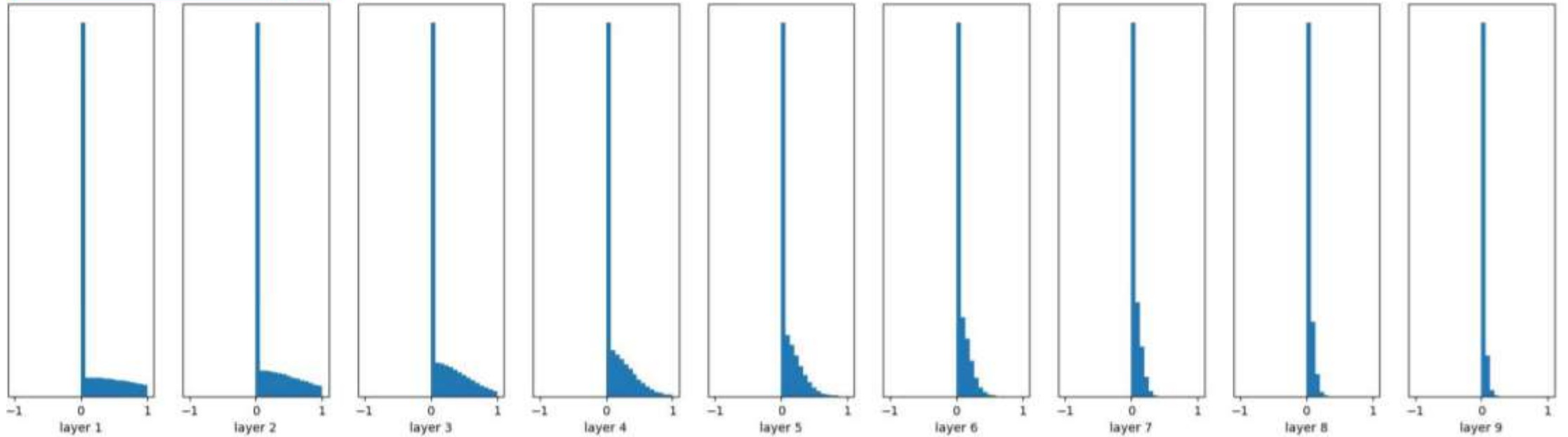
$$\text{Var}(w) = \frac{1}{n}$$



When using ReLU, output
close to zero again 😞

Xavier Initialization with ReLU

$$\text{Var}(w) = \frac{1}{n}$$

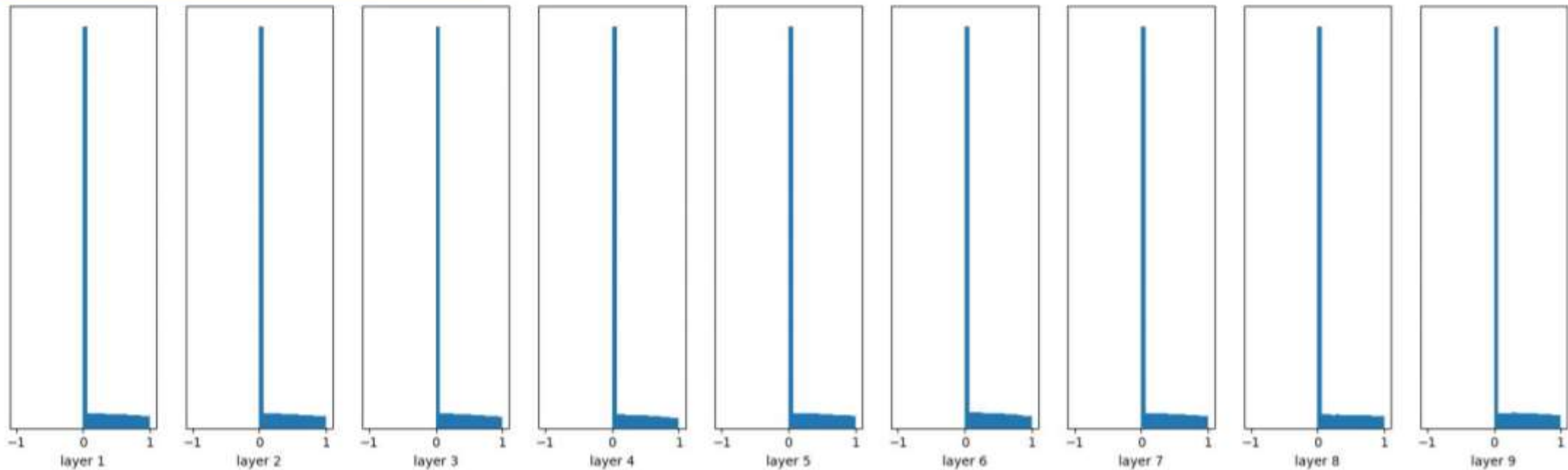


ReLU kills Half of the Data
What's the solution?

When using ReLU, output
close to zero again 😞

Xavier Initialization with ReLU

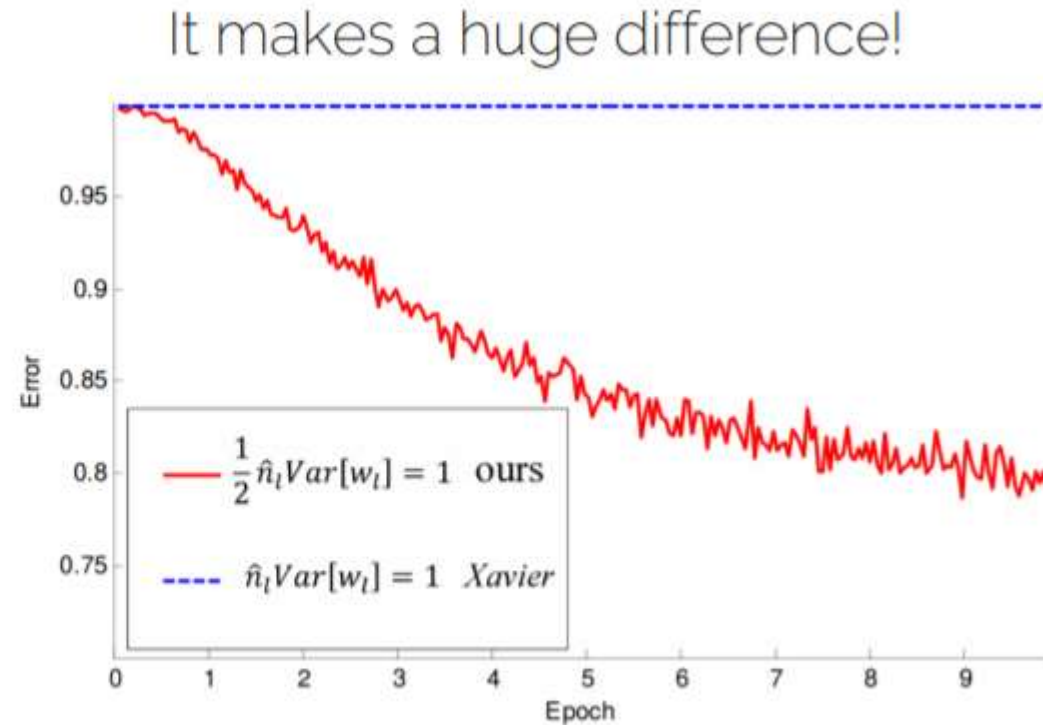
$$\text{Var}(w) = \frac{1}{n/2} = \frac{2}{n}$$



[He et al., ICCV'15] He Initialization

Xavier/2 Initialization with ReLU

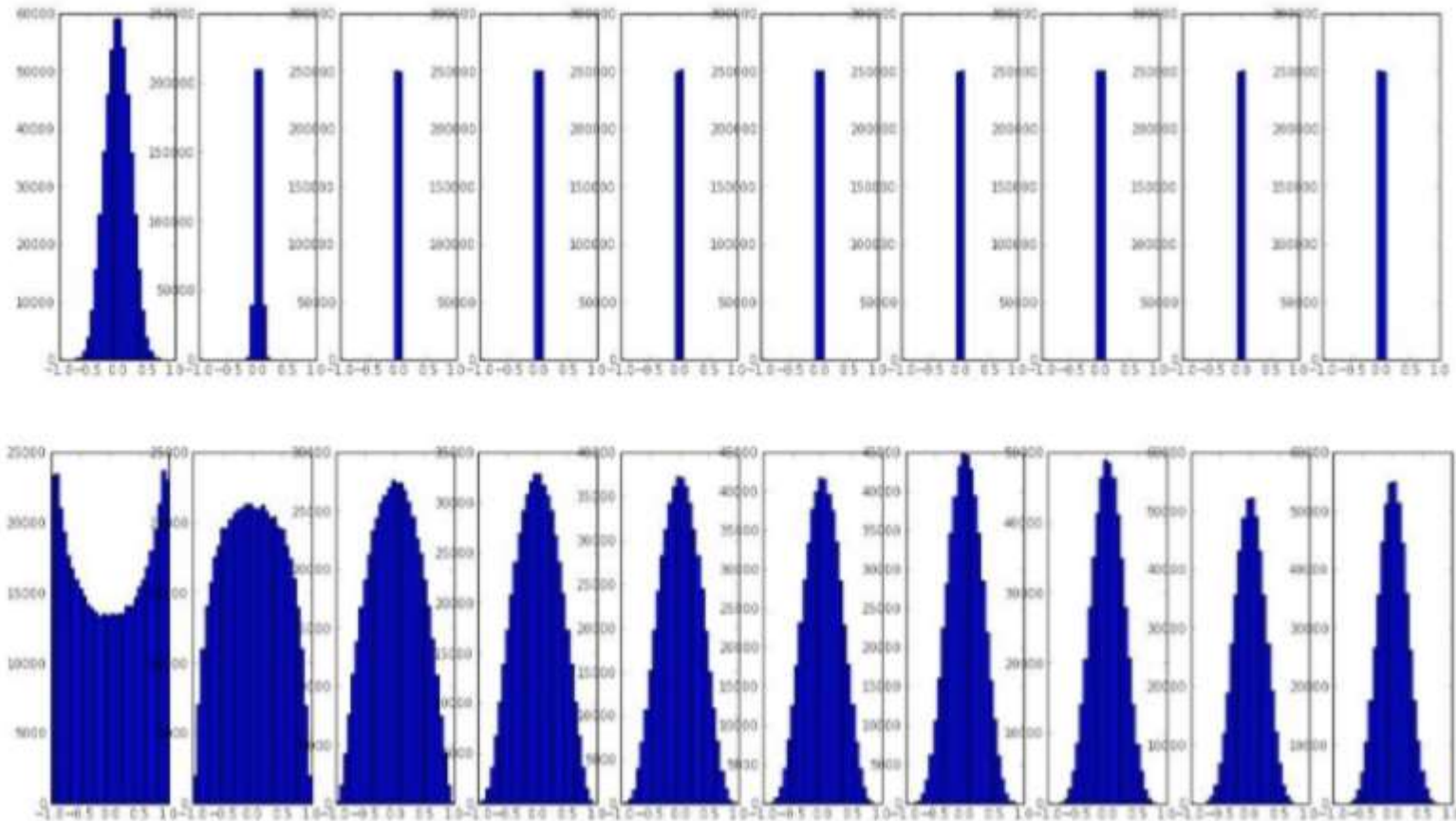
$$\text{Var}(w) = \frac{2}{n}$$



- Use ReLU and Xavier/2 initialization

Batch Normalization

All we want is
that our
activations do
not die out

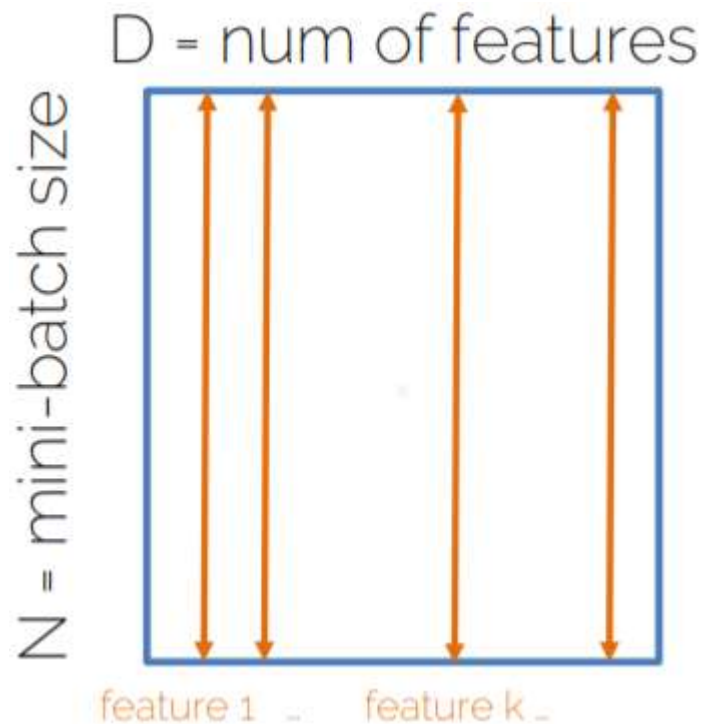


Batch Normalization

- Wish: Unit Gaussian activations (in our example)

Batch Normalization

- Wish: Unit Gaussian activations



Mean of your mini-batch
examples over feature k

$$\hat{\mathbf{x}}^{(k)} = \frac{\mathbf{x}^{(k)} - E[\mathbf{x}^{(k)}]}{\sqrt{\text{Var}[\mathbf{x}^{(k)}]}}$$

A blue arrow points from the text 'Mean of your mini-batch examples over feature k' to the term $E[\mathbf{x}^{(k)}]$ in the numerator of the equation.

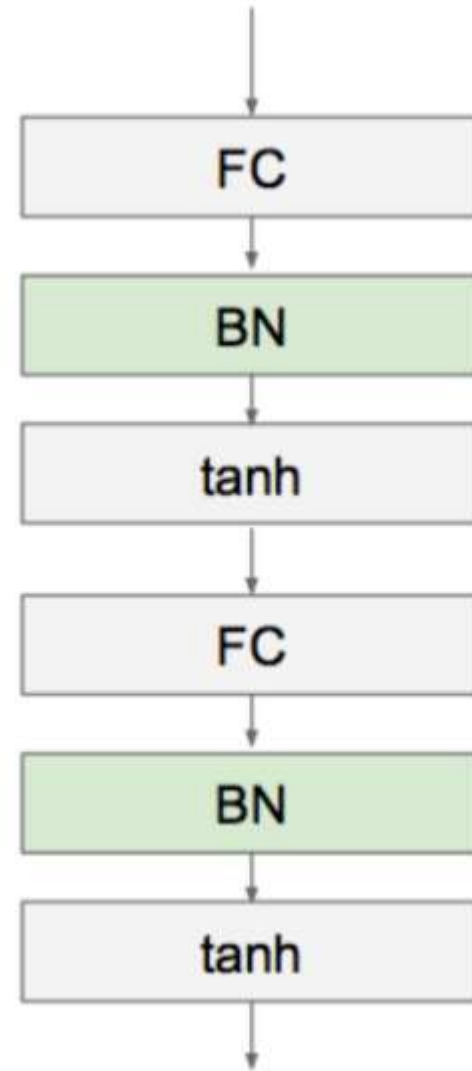
[Ioffe and Szegedy, PMLR'15] Batch Normalization

Batch Normalization

- In each dimension of the features, you have a unit gaussian
- BN normalizes the mean and variance of the inputs to your activation functions

BN Layer

A layer to be applied after Fully Connected (or Convolutional) layers and before non-linear activation functions



Batch Normalization

- Normalize

$$\hat{\mathbf{x}}^{(k)} = \frac{\mathbf{x}^{(k)} - E[\mathbf{x}^{(k)}]}{\sqrt{Var[\mathbf{x}^{(k)}]}}$$



Differentiable function so we can backprop through it...

- Allow the network to change the range

$$\mathbf{y}^{(k)} = \gamma^{(k)} \hat{\mathbf{x}}^{(k)} + \beta^{(k)}$$



These parameters will be optimized during backprop

Batch Normalization

- Normalize

$$\hat{\mathbf{x}}^{(k)} = \frac{\mathbf{x}^{(k)} - E[\mathbf{x}^{(k)}]}{\sqrt{Var[\mathbf{x}^{(k)}]}}$$

- Allow the network to change the range

$$\mathbf{y}^{(k)} = \gamma^{(k)} \hat{\mathbf{x}}^{(k)} + \beta^{(k)}$$

The network *can* learn to undo the normalization

$$\gamma^{(k)} = \sqrt{Var[\mathbf{x}^{(k)}]}$$

$$\beta^{(k)} = E[\mathbf{x}^{(k)}]$$

BN: Train vs Test

- Train time: mean and variance is taken over the minibatch
- Test-time: what happens if we can just process one image at a time?
 - No chance to compute a meaningful mean and variance

BN: Train vs Test

- Training: Compute mean and variance from mini-batch 1, 2, 3, ...
- Testing: Compute mean and variance by running an exponentially weighted averaged across training minibatches. Use them as σ_{test}^2 and μ_{test}

$$Var_{running} = \beta_m * Var_{running} + (1 - \beta_m) * Var_{minibatch}$$

$$\mu_{running} = \beta_m * \mu_{running} + (1 - \beta_m) * \mu_{minibatch}$$

β_m : momentum (hyperparameter)

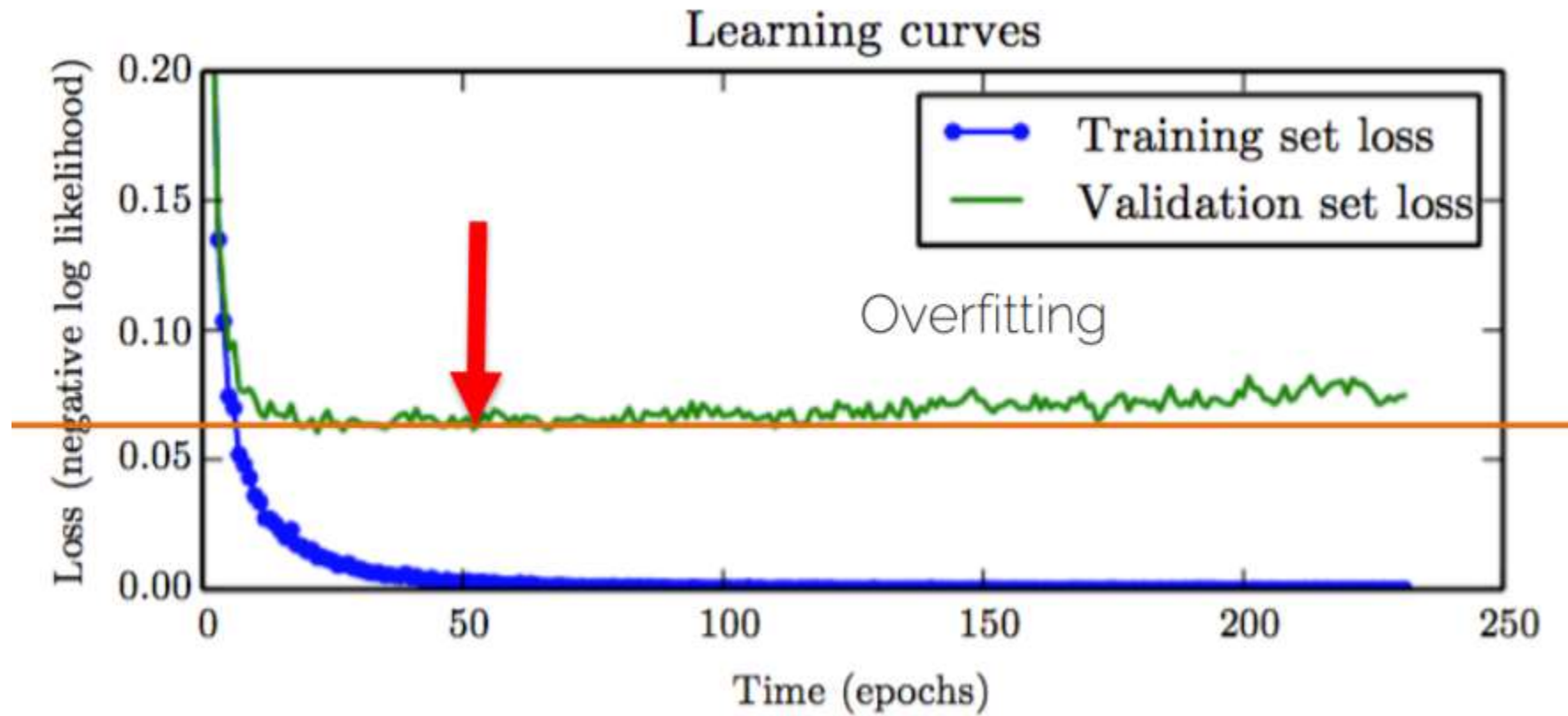
BN: What do you get?

- Very deep nets are much easier to train (more stable gradients)
- A much larger range of hyperparameters works similarly when using BN



Advanced Regularization

Early Stopping



Data Augmentation

A classifier has to be invariant to a wide variety of transformations



Data Augmentation

Helping the classifier: synthesize data simulating plausible transformations

a. No augmentation (= 1 image)



b. Flip augmentation (= 2 images)

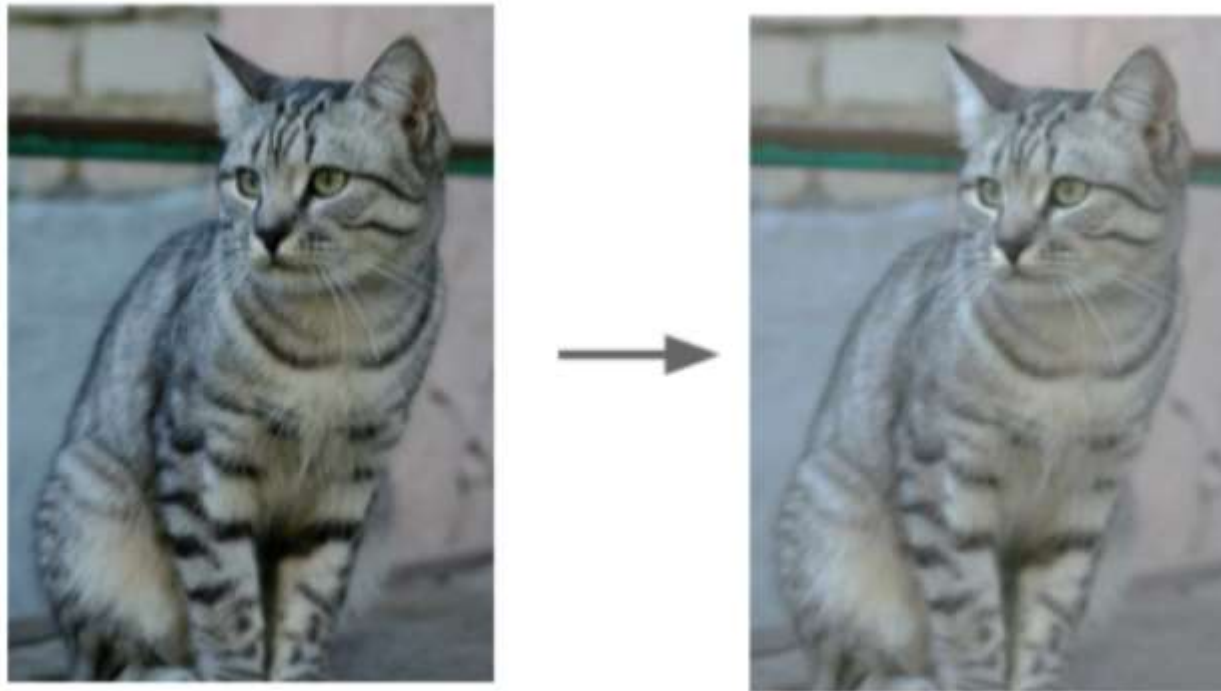


c. Crop+Flip augmentation (= 10 images)



Data Augmentation: Brightness

Random brightness and contrast changes



Data Augmentation

Random mix/combinations of :

- translation
- rotation
- stretching
- shearing,
- lens distortions,
- etc.

Data Augmentation

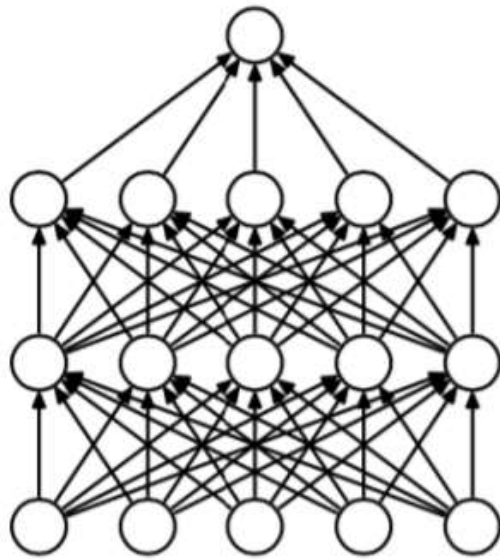
- When comparing two networks make sure to use the same data augmentation!
- Consider data augmentation a part of your network design

Bagging and Ensemble Methods

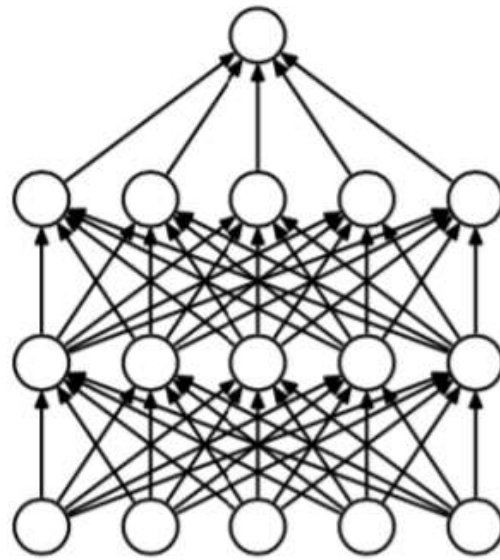
- Train multiple models and average their results
- E.g., use a different algorithm for optimization or change the objective function / loss function.
- If errors are uncorrelated, the expected combined error will decrease linearly with the ensemble size

Bagging and Ensemble Methods

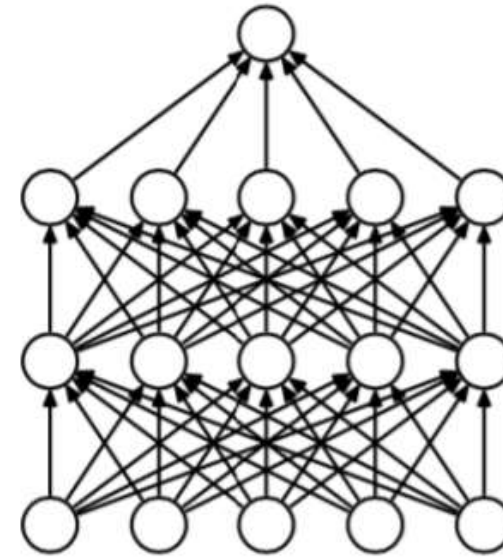
- Bagging: uses k different datasets



Training Set 1



Training Set 2

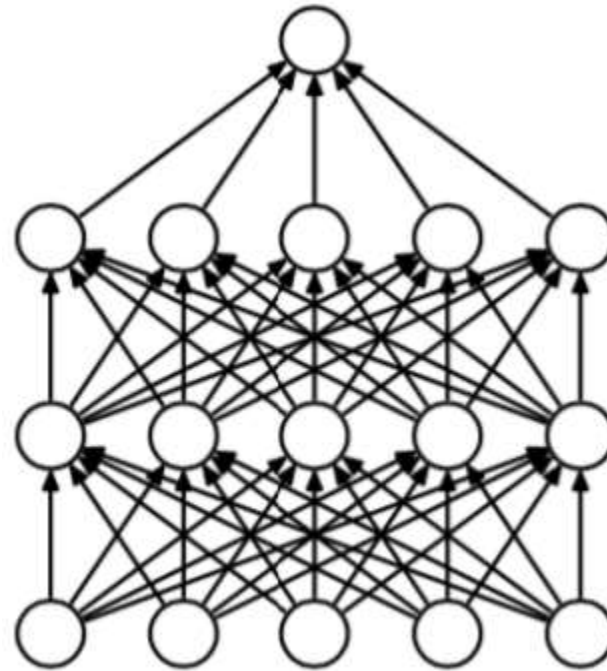


Training Set 3

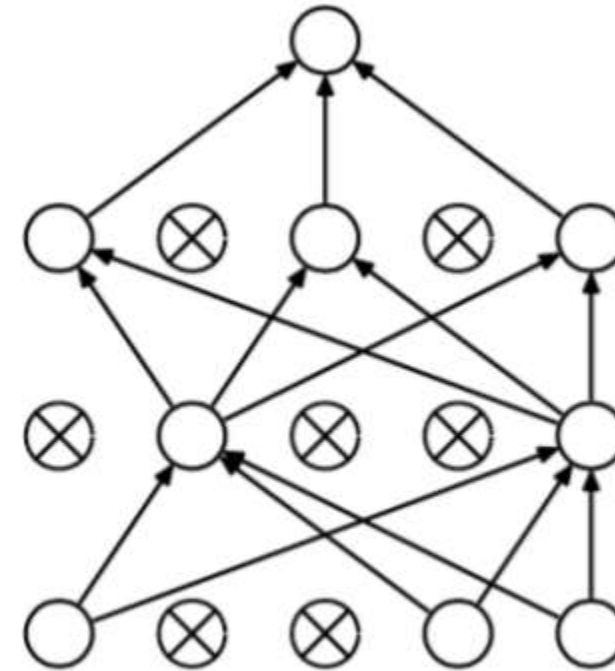
Image Source: [Srivastava et al., JMLR'14] Dropout

Dropout

Disable a random set of neurons (typically 50%)



(a) Standard Neural Net

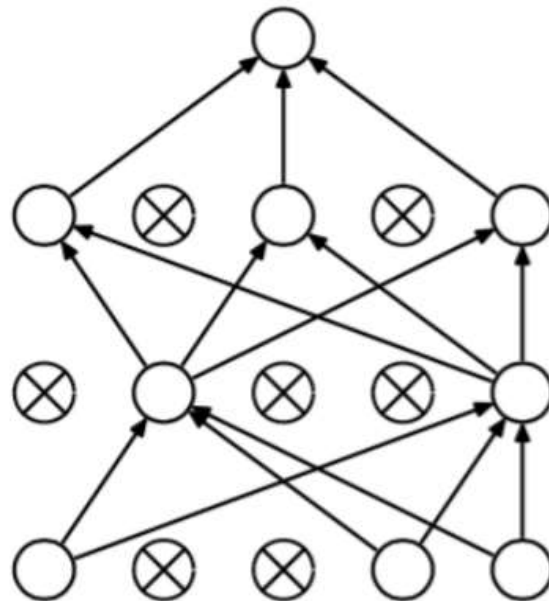


(b) After applying dropout.

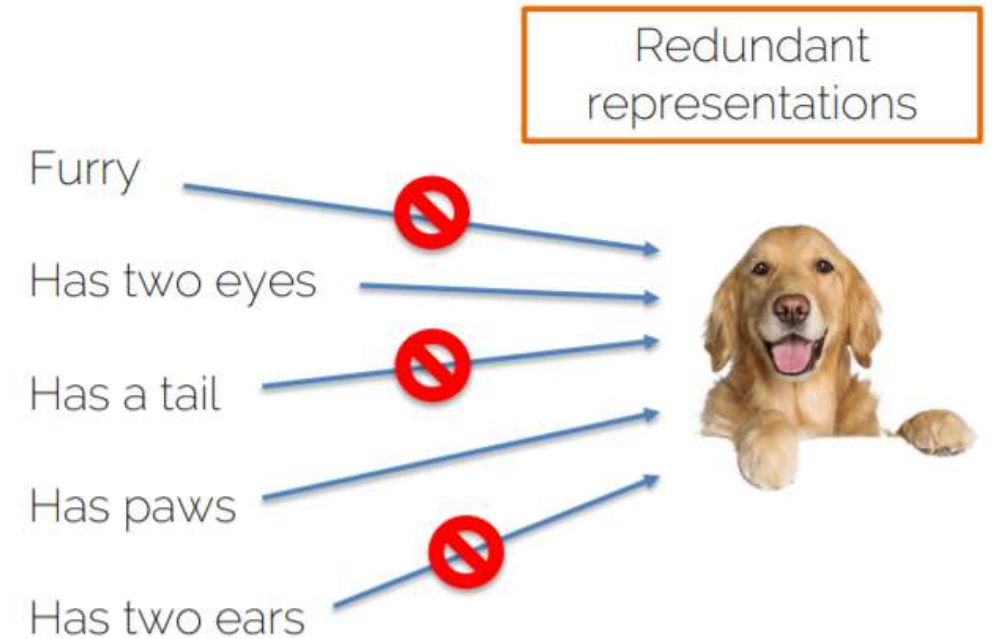
Image Source: [Srivastava et al., JMLR'14] Dropout

Dropout

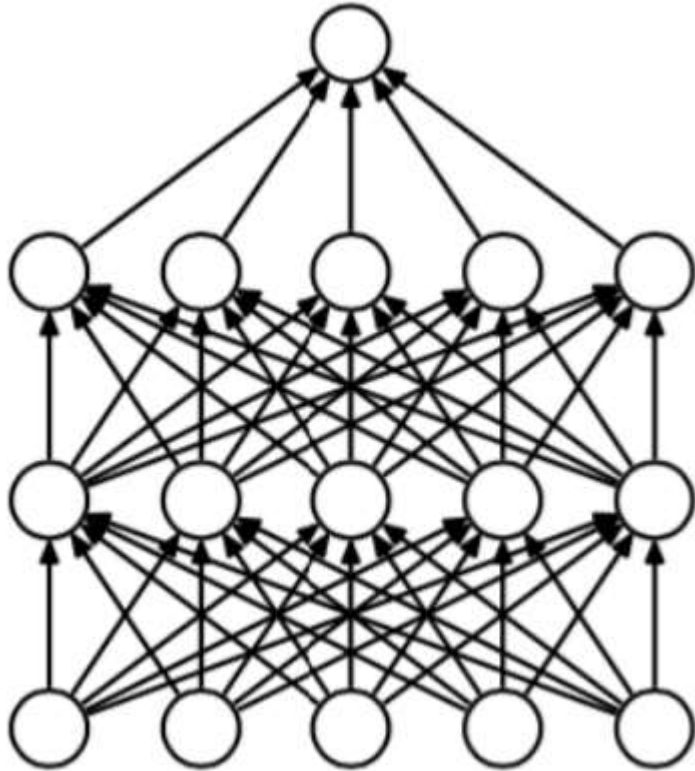
- Using half the network = half capacity
 - Redundant representations
 - Base your scores on more features
- Consider it as a model ensemble



(b) After applying dropout.



Dropout: Test Time



All neurons are “turned on” – no dropout

Conditions at train and test time are not the same

Dropout: Test Time

- Test:

$$z = (\theta_1 x_1 + \theta_2 x_2) \cdot p$$

Dropout probability
 $p = 0.5$

- Train:

